SYSTEM DESIGN INTERVIEW — MASTER HANDBOOK

> **For:** Product Companies | Big Tech | SDE Interviews | HLD Rounds | LLD Rounds | Senior Engineer Interviews

---

PART A — FOUNDATION NOTES

---

1. WHAT IS SYSTEM DESIGN?

Definition

System Design is the process of **defining the architecture, components, modules, interfaces, and data flow** of a system to satisfy specified functional and non-functional requirements.

What the Interviewer Evaluates

Evaluation Area	What They Look For
Problem understanding	Do you clarify before diving in?
Requirements thinking	Do you ask about scale, constraints, priorities?
Database decisions	Can you justify SQL vs NoSQL?
Architecture choices	Do you understand trade-offs?
Scalability	Can you scale to millions of users?
Reliability	Do you handle failures gracefully?
Communication	Can you explain your reasoning clearly?

Depth vs breadth	Can you go deep on any component when asked?

System Design vs Coding vs DSA

Aspect	DSA	Coding	System Design
Focus	Algorithms, data structures	Implementation	Architecture, trade-offs
Output	Working code	Compiled program	Design diagram + reasoning
Scale considered	Single machine	Single machine	Distributed, millions of users
Evaluated on	Correctness, complexity	Code quality	Judgment, trade-off reasoning

---

2. HLD — HIGH-LEVEL DESIGN

Definition

HLD (High-Level Design) defines the **overall system architecture** — what major components exist, how they interact, and what technologies are used — without going into implementation detail.

What HLD Focuses On

- **Services** → What microservices or modules exist
- **Databases** → Which database, how data is stored
- **APIs** → What endpoints exist, what they do
- **Network** → How components communicate

- **Load Balancer** → Distributes traffic
- **Cache** → What is cached and where
- **Message Queue** → Async communication between services
- **Storage** → File/object storage
- **External Services** → CDN, payment gateway, notifications

Simple HLD Architecture

```
[Client (Web / Mobile / API)]
          ↓
   [CDN] ■■■ for static assets
          ↓
   [Load Balancer]
          ↓
   [API Gateway] ← rate limiting, auth, routing
          ↓
 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
[Svc A] [Svc B] [Svc C]  [Svc N]
          ↓
 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
[Cache] [Database] [Queue] [Storage]
          ↓
   [Monitoring / Logging]
```

---

3. LLD — LOW-LEVEL DESIGN

Definition

LLD (Low-Level Design) defines the **internal implementation detail** of each component — classes, interfaces, methods, relationships, and design patterns.

What LLD Focuses On

- **Classes** → What objects exist
- **Interfaces** → What contracts are defined

- **Methods** → What operations each class performs
- **Relationships** → Inheritance, composition, association
- **Design Patterns** → Strategy, Factory, Observer, etc.
- **SOLID Principles** → Single responsibility, open-closed, etc.
- **Database Schema** → Tables, columns, relationships (entity level)

SOLID Principles (Quick Reference)

Principle	Full Name	Rule
S	Single Responsibility	One class = one reason to change
O	Open-Closed	Open for extension, closed for modification
L	Liskov Substitution	Subclass must be substitutable for base class
I	Interface Segregation	No class should depend on methods it doesn't use
D	Dependency Inversion	Depend on abstractions, not concrete implementations

---

4. HLD vs LLD

Feature	HLD	LLD
Goal	Define architecture	Define implementation

Abstraction Level	High (bird's eye view)	Low (component internals)
Main Focus	Services, DBs, APIs, scale	Classes, methods, patterns
Audience	Architects, product managers	Developers
Diagrams	Architecture diagram, data flow	Class diagrams, sequence diagrams
Design decisions	DB type, caching, messaging	Class hierarchy, design pattern
Interview style	"Design YouTube"	"Design a Parking Lot"

---

5. HOW TO APPROACH ANY SYSTEM DESIGN QUESTION

The Framework (Use This Every Time)

```
STEP 1  → Clarify the problem (2 min)
STEP 2  → Gather functional requirements
STEP 3  → Gather non-functional requirements
STEP 4  → Identify scale (users, RPS, data)
STEP 5  → Estimate traffic and storage
STEP 6  → Identify core entities
STEP 7  → Choose the database
STEP 8  → Design database schema
STEP 9  → Decide indexes, partitioning, replication
STEP 10 → Design APIs
STEP 11 → Design high-level architecture
STEP 12 → Add cache
STEP 13 → Add messaging / async processing
```

```
STEP 14 → Handle networking decisions
STEP 15 → Handle failures
STEP 16 → Handle security
STEP 17 → Discuss scalability
STEP 18 → Discuss trade-offs
STEP 19 → Handle follow-up questions
```

What to Do at Each Step

Step 1 — Clarify:

> "Before I start designing, I have a few questions. Who are the users? What platforms do we support? Are there existing systems? What is the most important feature to get right?"

Step 2 — Functional Requirements:

> Core features only. What the system MUST do. What a user can do. Strip it to essentials.

Step 3 — Non-Functional Requirements:

> Latency, availability, consistency, durability, throughput, security. Ask: "What is more important — consistency or availability for this use case?"

Step 4 — Scale:

> "Roughly how many users do we have? What's the DAU? Is it read-heavy or write-heavy?"

Step 5 — Estimate:

> Back-of-envelope math. RPS = DAU × actions/day / 86,400. Storage = objects × size × time.

Step 6 — Core Entities:

> Name the key data objects: User, Post, Message, Order, Product, etc.

Step 7 — Database Choice:

> Most important decision. Justify SQL vs NoSQL based on data structure, query patterns, scale, and consistency needs.

Step 8 — Schema:

> Table names, key columns, data types. Don't over-design — sketch the critical tables.

Step 9 — DB Scaling:

> Read replicas? Sharding? Which column to shard on? Index on which columns?

Step 10 — APIs:

> REST endpoints. Method, path, request body, response. Keep it clean.

Step 11 — HLD:

> Draw the architecture. Name each component. Explain data flow.

Step 12 — Cache:

> What is expensive to compute or fetch? Add Redis/Memcached. Pick strategy.

Step 13 — Async:

> What operations can be decoupled? Add Kafka/RabbitMQ where writes or processing should not block the user.

Step 14 — Networking:

> HTTP or WebSocket? REST or gRPC? CDN for static? DNS strategy? Load balancer algorithm?

Step 15 — Failures:

> "What if X fails?" — redundancy, retry, circuit breaker, failover.

Step 16 — Security:

> Auth (JWT/OAuth), HTTPS, rate limiting, input validation, encryption at rest.

Step 17 — Scalability:

> How does each component scale independently? Which is the bottleneck?

Step 18 — Trade-offs:

> State every major decision and its trade-off explicitly.

Step 19 — Follow-ups:

> Expect: "What if traffic is 10x?" / "What if DB fails?" / "Why not NoSQL?"

---

6. REQUIREMENTS

Functional Requirements

What the system **must do** — user-visible features.

- Core user actions (create, read, update, delete)
- Core business operations (order, pay, deliver)
- APIs the system must expose
- Data the system must store and retrieve

How to identify: Ask "What can a user do with this system?"

Non-Functional Requirements

Requirement	Description	How to Measure
Scalability	Handle growing users/data	Users, RPS
Availability	System is up and accessible	99.9% = 8.7h downtime/year
Reliability	System behaves correctly	MTBF, error rate
Consistency	All users see same data	Strong vs eventual
Durability	Data is not lost	Replication factor
Latency	Response time	p50, p99 in ms
Throughput	Requests handled per second	RPS
Security	Data is protected	Auth, encryption
Fault Tolerance	Works despite failures	MTTR, redundancy

Maintainability	Easy to modify/deploy	Modular design

How to identify: Ask "What happens if ___? What are the constraints?"

Availability Numbers

Availability	Downtime/Year	Downtime/Month
99%	3.65 days	7.3 hours
99.9%	8.7 hours	43.8 minutes
99.99%	52.6 minutes	4.4 minutes
99.999%	5.26 minutes	26.3 seconds

---

7. SCALE ESTIMATION

Key Metrics

Metric	Description
DAU	Daily Active Users
MAU	Monthly Active Users
RPS	Requests Per Second
Peak RPS	Peak = 2–3× average RPS
Read/Write Ratio	e.g., 100:1 for Twitter

Data per object	e.g., 1 tweet = ~500 bytes

Formulas

```
RPS = DAU × (actions per user per day) / 86,400

Storage/day = writes/day × avg_object_size

Storage/year = storage/day × 365 × replication_factor

Bandwidth = RPS × avg_response_size
```

Example — Twitter-like System

```
DAU = 100 million users
Avg 10 actions/day (read/write mixed)
Read/Write ratio = 100:1 → 99% reads

Total RPS = 100M × 10 / 86,400 ≈ 11,500 RPS
Write RPS ≈ 115 RPS (1%)
Read RPS ≈ 11,385 RPS (99%)

Tweet = ~500 bytes
Writes/day = 115 × 86,400 ≈ 10M tweets/day
Storage/day = 10M × 500B = 5 GB/day
Storage/year = 5 × 365 ≈ 1.8 TB/year (× replication ≈ 5 TB)
```

Powers of 10 (Memorize)

Value	Approx
1 million	10■
1 billion	10■

1 KB	10^3 bytes
1 MB	10■ bytes
1 GB	10■ bytes
1 TB	10^{12} bytes
1 day	86,400 seconds
1 year	~31.5M seconds

---

8. CORE SYSTEM DESIGN COMPONENTS

Client

- **What:** Browser, mobile app, desktop client that initiates requests.
- **Why:** Entry point of all user interactions.
- **Trade-off:** Thin client (more server work) vs thick client (more client work, offline support).

API Gateway

- **What:** Single entry point for all client requests; routes to appropriate backend service.
- **Why:** Centralizes auth, rate limiting, SSL termination, logging, routing.
- **When:** Always in microservices architecture.
- **Trade-off:** Single point of failure if not replicated; adds latency; can become a bottleneck.

Load Balancer

- **What:** Distributes incoming traffic across multiple servers.

- **Why:** Prevents any single server from being overwhelmed; enables horizontal scaling.
- **Algorithms:** Round Robin, Least Connections, IP Hash, Weighted Round Robin.
- **Types:** L4 (TCP level) and L7 (HTTP level — can route by URL/headers).
- **Trade-off:** L7 is smarter but more compute-intensive than L4.

Application Server

- **What:** Processes business logic; handles requests from load balancer.
- **Why:** Stateless servers are horizontally scalable.
- **When:** Always — the core of any backend.
- **Trade-off:** Should be stateless (session in cache/DB, not on server).

Microservice

- **What:** Small, independently deployable service that handles one domain.
- **Why:** Independent scaling, deployment, failure isolation.
- **When:** When teams are large, domains are distinct, and services need independent scaling.
- **Trade-off:** Adds network overhead, distributed system complexity, harder debugging.

Monolith

- **What:** Single deployable unit containing all business logic.
- **Why:** Simple to develop, deploy, debug for small teams.
- **When:** Early stage, small team, fast iteration required.
- **Trade-off:** Hard to scale specific parts; one crash affects everything.

Database

- **What:** Persistent storage for structured or unstructured data.
- **Why:** Core of every data-driven system.

- **When:** Always.
- **Trade-off:** SQL vs NoSQL, consistency vs availability, latency vs durability.

Cache

- **What:** Fast in-memory store (Redis, Memcached) for frequently accessed data.
- **Why:** Reduces database load; dramatically lowers read latency.
- **When:** Read-heavy systems, computed data, session storage.
- **Trade-off:** Data can be stale; cache invalidation is hard; additional infrastructure.

Message Queue

- **What:** Buffer between producer and consumer for async communication.
- **Why:** Decouples services; handles traffic spikes; enables retry.
- **When:** When operations don't need to be synchronous (email, notifications, analytics).
- **Trade-off:** Adds latency; message ordering complexity; at-least-once delivery may cause duplicates.

Message Broker

- **What:** Software (Kafka, RabbitMQ) that manages message queues and routing.
- **Why:** Routes messages between producers and consumers reliably.
- **Trade-off:** Kafka = high-throughput, durable log; RabbitMQ = flexible routing, simpler.

Object Storage

- **What:** Stores binary objects (images, videos, files) at massive scale — S3, GCS.
- **Why:** Cheap, durable, infinitely scalable blob storage.
- **When:** Any file/media/document storage need.
- **Trade-off:** Not for structured queries; higher latency than database; no ACID.

CDN (Content Delivery Network)

- **What:** Geographically distributed servers that cache and serve static content close to users.
- **Why:** Reduces latency; reduces origin server load.
- **When:** Static assets (images, JS, CSS, videos), global user base.
- **Trade-off:** Cache invalidation is slow; cost per request; stale content risk.

Search Engine

- **What:** Elasticsearch, OpenSearch — full-text and complex query search.
- **Why:** Databases are slow for full-text search; search engines use inverted indexes.
- **When:** Product search, log search, full-text search.
- **Trade-off:** Eventual consistency with primary DB; additional sync needed; extra infra.

Notification Service

- **What:** Sends push notifications, emails, SMS to users.
- **Why:** Decoupled async delivery of user alerts.
- **When:** Any user-facing event (order placed, message received).
- **Trade-off:** Delivery is not guaranteed; need retry; rate limiting by carrier.

Rate Limiter

- **What:** Controls how many requests a client can make in a time window.
- **Why:** Prevents abuse, DDoS, API overload.
- **Algorithms:** Token Bucket, Leaky Bucket, Fixed Window, Sliding Window.
- **When:** Always on public APIs.
- **Trade-off:** Can block legitimate traffic if misconfigured; adds latency.

Reverse Proxy

- **What:** Server that sits in front of backend servers, forwards client requests.
- **Why:** SSL termination, caching, load balancing, security.
- **When:** Nginx, HAProxy — almost always in production.
- **Trade-off:** Additional hop; single point of failure if not replicated.

Service Discovery

- **What:** Mechanism to find which instances of a service are available (Consul, Eureka, K8s DNS).
- **Why:** In microservices, service instances come and go dynamically.
- **When:** Microservices with dynamic scaling.
- **Trade-off:** Added complexity; discovery delay; stale registry risk.

Distributed Lock

- **What:** Ensures only one process performs a critical operation at a time (Redis SETNX, ZooKeeper).
- **Why:** Prevent race conditions in distributed systems.
- **When:** Inventory deduction, payment processing, leader election.
- **Trade-off:** Lock expiry edge cases; network partition risks (Redlock controversy).

Monitoring & Logging

- **What:** Tracks system health (Prometheus, Grafana) and records events (ELK Stack).
- **Why:** Observability — find bugs, performance issues, anomalies.
- **When:** Always in production.
- **Trade-off:** Log storage cost; alert fatigue if misconfigured.

---

9. DATABASES — DEEP DIVE

SQL / Relational Databases

Key Concepts

Concept	Description
Table	Structured data in rows and columns
Primary Key	Unique identifier for each row
Foreign Key	Reference to primary key in another table
Index	B-tree structure for fast lookups
Composite Index	Index on multiple columns
Transaction	Group of operations that succeed or fail together
ACID	Atomicity, Consistency, Isolation, Durability
Normalization	Remove redundancy; split data into multiple tables
Denormalization	Add redundancy for faster reads (join avoidance)
Joins	Combine data from multiple tables

ACID Properties

Property	Meaning

Atomicity	All operations in transaction succeed or all are rolled back
Consistency	DB moves from one valid state to another
Isolation	Concurrent transactions don't interfere
Durability	Committed data is persisted even after crash

NoSQL Databases

Type	Description	Examples	Best For
Key-Value	Simple key → value lookup	Redis, DynamoDB	Sessions, caching, simple lookup
Document	JSON/BSON documents	MongoDB, Firestore	Flexible schemas, nested data
Wide-Column	Column families, row key	Cassandra, HBase	High write throughput, time-series
Graph	Nodes and edges	Neo4j, Amazon Neptune	Social graphs, recommendations

When to Use NoSQL

- Schema is flexible/frequently changing
- Horizontal scaling is needed
- Write throughput is very high (Cassandra)

- Simple key-based lookups (Redis)
- No complex joins needed
- Eventual consistency is acceptable

Database Selection Guide

Requirement	Best Database	Why
Financial transactions	**PostgreSQL**	ACID, strong consistency
User profiles, sessions	**MongoDB**	Flexible schema, document model
High-write time-series	**Cassandra**	Append-only, wide-column, fast writes
Caching / sessions	**Redis**	In-memory, sub-millisecond latency
Full-text search	**Elasticsearch**	Inverted index, fast text search
Social graph	**Neo4j**	Graph traversal is native
Simple key lookup at scale	**DynamoDB**	Serverless, auto-scaling, single-digit ms
Analytics / OLAP	**Redshift / BigQuery**	Columnar storage, batch queries
General web app	**MySQL / PostgreSQL**	Mature, reliable, well-supported

How an Interviewer Expects You to Choose

1. **What is the data model?** (Relational → SQL; Nested/flexible → NoSQL)
2. **What are the query patterns?** (Complex joins → SQL; Key lookups → NoSQL)
3. **What are the consistency requirements?** (Strong → SQL/PostgreSQL; Eventual okay → Cassandra/DynamoDB)
4. **What is the write throughput?** (Very high → Cassandra; Moderate → PostgreSQL)
5. **Do you need full-text search?** (Yes → Elasticsearch alongside main DB)
6. **Do you need transactions?** (Yes → SQL; No → NoSQL more flexible)
7. **What is the scale?** (Petabytes → NoSQL/data warehouse; Terabytes → SQL fine with sharding)

Scaling Databases

Read Replicas

- Add **read-only copies** of the primary database.
- All writes go to primary; all reads go to replicas.
- Reduces load on primary; increases read throughput.
- **Trade-off:** Replication lag → reads may see slightly stale data.

Sharding (Horizontal Partitioning)

- Split data across **multiple database servers** (shards) by a shard key.
- Each shard holds a subset of data.
- **Good shard keys:** User ID, geographic region, hash-based.
- **Bad shard keys:** Low-cardinality fields (e.g., gender) → hot partitions.
- **Trade-off:** Cross-shard queries are complex; resharding is painful.

Partitioning

- **Horizontal partitioning (sharding):** Split rows across servers.

- **Vertical partitioning:** Split columns into separate tables (rarely accessed columns separated).
- **Range partitioning:** Partition by range of key values (e.g., date ranges).
- **Hash partitioning:** Shard key hashed to determine partition.

Consistent Hashing

- Ring-based hashing where adding/removing nodes only rebalances a small portion of data.
- Used in Cassandra, DynamoDB, distributed caches.
- **Trade-off:** Can still create hot spots with small rings; virtual nodes mitigate this.

Read/Write Separation

```
Write → Primary DB
Read  → Read Replicas (multiple)
```

- Effective when read:write ratio is high (e.g., 90:10 or more reads).

Strong vs Eventual Consistency

Type	Description	Use Case
Strong Consistency	All reads see latest write	Banking, inventory, orders
Eventual Consistency	Reads may be slightly stale but converge	Social feeds, likes, analytics

Connection Pooling

- Reuse existing DB connections instead of creating new ones per request.
- Reduces connection overhead; DB has limited connections.
- Tools: PgBouncer (PostgreSQL), HikariCP (Java), connection pool in ORMs.

---

10. CACHING

Why Caching is Needed

- Database queries are slow (disk I/O, network).
- Same data is read thousands of times.
- Computed/aggregated data is expensive to recalculate.

Cache Strategies

Strategy	Description	Use When
Cache-Aside (Lazy Loading)	App checks cache first; on miss, fetches from DB and stores in cache	Most common; read-heavy systems
Read-Through	Cache sits in front of DB; cache fetches from DB on miss	When cache vendor handles DB fetching
Write-Through	Every write goes to cache AND DB synchronously	When consistency between cache and DB is required
Write-Back (Write-Behind)	Write to cache only; async write to DB later	High write throughput; risk of data loss
Write-Around	Write directly to DB; skip cache	Data written once and rarely re-read

Cache-Aside Flow

```
App reads data:
 ↓ Check cache → HIT → return cached data
```

```
 ↓ Check cache → MISS → query DB → store in cache → return data

App writes data:
 → Update DB
 → Invalidate cache entry (or update cache)
```

Cache Eviction Policies

Policy	Rule
LRU (Least Recently Used)	Remove least recently accessed item
LFU (Least Frequently Used)	Remove least frequently accessed item
FIFO	Remove oldest item first
TTL (Time to Live)	Remove after fixed expiry time

Cache Problems

Problem	Description	Solution
Cache Stampede	Cache expires → many requests hit DB simultaneously	Mutex lock / probabilistic early expiry / staggered TTL
Cache Penetration	Requests for non-existent keys bypass cache, hammer DB	Cache null values; Bloom filter to check existence first
Cache Avalanche	Many cache entries expire at once → DB overwhelmed	Jitter TTL (add random offset); gradual expiry; persistent cache

Hot Keys	Very few keys get extreme traffic	Replicate hot keys across multiple cache nodes; local in-memory cache

Distributed Cache

- Redis Cluster or Memcached cluster.
- Data partitioned across nodes using consistent hashing.
- Supports replication for availability.

CDN Cache

- Caches static assets (images, videos, JS, CSS) at edge nodes close to users.
- TTL-based expiry; cache invalidation via CDN purge APIs.
- Origin server only serves uncached or invalidated content.

---

11. MESSAGING & ASYNC SYSTEMS

Synchronous vs Asynchronous

Type	Description	Use When
Synchronous	Caller waits for response	Read data, critical operations, immediate feedback
Asynchronous	Caller doesn't wait; result comes later	Email, notifications, heavy processing, decoupling

Message Queue vs Message Broker

- **Message Queue:** FIFO queue. Simple. One consumer per message. (SQS, RabbitMQ basic queue)

- **Message Broker:** Routes messages, supports pub/sub, multiple consumers. (Kafka, RabbitMQ with exchanges)

Kafka

- **Log-based message broker** — messages stored as an ordered, persistent log.
- **Partitions** → Topic split into partitions; each partition ordered.
- **Consumer Groups** → Each group gets all messages; partition assigned to one consumer in group.
- **Offset** → Position of a consumer in a partition.
- **Retention** → Messages kept for configurable time (even after consumption).
- **Best for:** High throughput, audit logs, event sourcing, data pipelines.

RabbitMQ

- **Traditional message broker** with flexible routing.
- Messages deleted after consumption.
- Supports **exchanges** (direct, fanout, topic, headers).
- **Best for:** Task queues, complex routing, RPC patterns.

Delivery Guarantees

Guarantee	Description	Risk
At-most-once	Message sent once; may be lost	Data loss possible
At-least-once	Message retried until ACKed; may be duplicate	Duplicate processing possible
Exactly-once	Message processed exactly once	Complex, high overhead

Idempotency

- An operation that produces the **same result when applied multiple times**.
- Critical for at-least-once delivery — if duplicate messages arrive, processing same message twice must be safe.
- Implement via unique message ID + deduplication check before processing.

Dead Letter Queue (DLQ)

- Messages that **repeatedly fail processing** are moved to a DLQ.
- Prevents failed messages from blocking the queue.
- Used to inspect, debug, and reprocess failed messages.

---

12. DISTRIBUTED SYSTEMS

CAP Theorem

> A distributed system can guarantee at most **two** of the three:

Property	Description
Consistency (C)	All nodes see the same data at the same time
Availability (A)	Every request gets a response (not necessarily latest data)
Partition Tolerance (P)	System continues to function despite network partitions

> **P is not optional** in real distributed systems (networks always partition). So the real choice is **CP vs AP**.

System	Choice	Examples

CP	Consistent but may be unavailable during partition	HBase, ZooKeeper, MongoDB (strong mode)
AP	Available but may return stale data	Cassandra, DynamoDB, CouchDB

Replication

- **Master-Slave (Primary-Replica):** Writes to primary; async/sync replicated to slaves.
- **Multi-Master:** Multiple write nodes; conflict resolution needed.
- **Synchronous Replication:** Write confirmed only after replica acknowledges → strong consistency, higher latency.
- **Asynchronous Replication:** Write confirmed immediately; replica updates later → lower latency, eventual consistency.

Distributed Transactions

- **2PC (Two-Phase Commit):** Coordinator asks all participants to prepare, then commit. Blocks if coordinator fails.
- **Saga Pattern:** Sequence of local transactions with compensating transactions on failure. Preferred in microservices.

Leader Election

- One node becomes the "leader" to coordinate decisions.
- Used for: primary database election, distributed lock management.
- Algorithms: Raft, Paxos, Bully algorithm.
- Tools: ZooKeeper, etcd.

Fault Tolerance

- **Retry with Exponential Backoff:** Retry on failure; double wait time each attempt; add jitter.

- **Circuit Breaker:** After N consecutive failures, stop calling the failing service. Reset after timeout.
- **Timeout:** Don't wait forever; fail fast if service doesn't respond.
- **Bulkhead:** Isolate resources (thread pools, connection pools) per service; failure in one doesn't exhaust resources for others.

---

13. RELIABILITY & FAILURE HANDLING

Key Concepts

Concept	Description
SPOF	Single Point of Failure — any component whose failure takes down the system
Redundancy	Duplicate critical components to eliminate SPOF
Failover	Automatically switch to backup when primary fails
Health Check	Periodic ping to verify component is alive
Retry	Retry failed requests with backoff
Circuit Breaker	Stop calling failing service to prevent cascade failures
Graceful Degradation	Return partial/cached results instead of error when a dependency fails

Multi-Region Architecture

Pattern	Description
Active-Passive	One region active; another in standby; failover on failure
Active-Active	Both regions serve traffic; globally distributed; more complex

Disaster Recovery

Term	Description
RPO (Recovery Point Objective)	Max acceptable data loss (how old can restored data be?)
RTO (Recovery Time Objective)	Max acceptable downtime (how fast must system recover?)

Failure Handling Patterns

```
Circuit Breaker States:
 CLOSED → calls pass through normally
 OPEN → all calls fail immediately (no attempt to call failing service)
 HALF-OPEN → allow limited calls to test if service recovered
```

---

14. NETWORKING FOR SYSTEM DESIGN

Protocols and Their Role

Protocol	Layer	Role in System Design

IP	Network	Routing packets between services
TCP	Transport	Reliable delivery (HTTP, DB connections)
UDP	Transport	Low latency (gaming, video, DNS)
HTTP/1.1	Application	Standard web API; keep-alive connections
HTTP/2	Application	Multiplexed; better for many parallel requests
HTTPS/TLS	Application	Encrypted communication
WebSocket	Application	Full-duplex real-time communication
gRPC	Application	Efficient binary RPC for microservice communication

Communication Patterns

Pattern	How	Use Case
Request-Response (REST)	Client requests, server responds	Most web APIs
WebSocket	Persistent bidirectional connection	Chat, live updates, gaming

Long Polling	Client keeps connection open; server responds when event occurs	Simple real-time without WebSocket
Server-Sent Events (SSE)	Server streams events to client (one-way)	Live feeds, dashboards
gRPC	Binary protocol, bidirectional streaming	Microservice-to-microservice

Connection Pooling

- DB connection creation is expensive; pool reuses existing connections.
- Application server maintains a pool; requests borrow and return connections.

Keep-Alive

- HTTP keep-alive reuses TCP connection for multiple requests.
- Avoids TCP 3-way handshake overhead for every request.

DNS in System Design

- **DNS-based load balancing:** Multiple IPs for same domain (Round Robin DNS).
- **DNS failover:** Switch DNS record to backup server on failure.
- **TTL considerations:** Low TTL = faster failover; High TTL = cached longer (less DNS load).

CDN

- Static assets cached at edge nodes globally.
- Reduces latency for global users.
- Reduces origin server load by 80–90% for media-heavy apps.

Network Latency Targets

Network Hop	Typical Latency
In-datacenter	< 1 ms
Same region	1–5 ms
Cross-region (continent)	50–150 ms
Cross-continent	100–300 ms
Disk seek	~10 ms
Redis (same DC)	< 1 ms

---

15. LOAD BALANCING

Algorithms

Algorithm	Description	Best For
Round Robin	Distribute requests evenly in turn	Homogeneous servers
Weighted Round Robin	Servers with more capacity get more traffic	Heterogeneous servers
Least Connections	Send to server with fewest active connections	Long-lived connections
IP Hash	Route by client IP (sticky sessions)	Session affinity needed

Random	Random server selection	Simple, works well with many servers

L4 vs L7 Load Balancing

Feature	L4 (Transport)	L7 (Application)
Routes based on	IP, TCP port	URL, headers, cookies
Understands HTTP	No	Yes
SSL termination	No	Yes
Speed	Faster	Slightly slower
Examples	HAProxy, AWS NLB	Nginx, AWS ALB

---

16. SECURITY

Authentication vs Authorization

Concept	Question	Example
Authentication	Who are you?	Login with username/password
Authorization	What can you do?	Admin can delete; user can only read

JWT (JSON Web Token)

- Signed token containing user claims.

- Stateless — server doesn't store session.
- Structure: `Header.Payload.Signature`
- **Risk:** Cannot be revoked before expiry (use short TTL + refresh tokens).

OAuth 2.0

- Delegated authorization — "Login with Google."
- User authorizes third-party app to access resources on their behalf.
- Flows: Authorization Code (web apps), Client Credentials (service-to-service), Implicit (deprecated).

Session-Based Authentication

- Server stores session in memory/DB/Redis.
- Client holds Session ID (cookie).
- Stateful — server must look up session on every request.
- Easy to revoke (delete session from store).

HTTPS/TLS

- All communication encrypted in transit.
- Certificates from trusted CA.
- TLS 1.3 preferred.

Rate Limiting

- **Token Bucket:** Tokens added at fixed rate; each request consumes a token.
- **Leaky Bucket:** Requests queued; processed at fixed rate.
- **Fixed Window:** Max N requests per time window.
- **Sliding Window:** Smoother version of fixed window.

RBAC (Role-Based Access Control)

- Users assigned roles (Admin, Editor, Viewer).

- Roles granted permissions.
- User → Role → Permissions.

Data Protection

- **Encryption at rest:** Data encrypted on disk (AES-256).
- **Encryption in transit:** TLS for all network communication.
- **PII handling:** Hash or encrypt sensitive data (passwords with bcrypt, PII with AES).

---

17. LLD DESIGN PATTERNS

CREATIONAL PATTERNS

Singleton

- **Problem:** Ensure only ONE instance of a class exists globally.
- **Structure:**

```
class Singleton:
   _instance = None
   @staticmethod
   def get_instance():
       if not _instance: _instance = Singleton()
       return _instance
```

- **When to use:** Database connection pool, config manager, logger.
- **SD Application:** Logger, Config Service, DB Connection Pool.

Factory Method

- **Problem:** Create objects without specifying the exact class.

- **Structure:** Creator defines a factory method; subclasses override it to return specific product.
- **When to use:** When the type of object to create is determined at runtime.
- **SD Application:** Payment gateway (CreditCardPayment, UPIPayment, NetBankingPayment).

Abstract Factory

- **Problem:** Create families of related objects without specifying concrete classes.
- **When to use:** UI toolkit (WindowsButton + WindowsScrollbar vs MacButton + MacScrollbar).
- **SD Application:** Multi-platform notification (EmailNotification factory vs SMSNotification factory).

Builder

- **Problem:** Construct complex objects step by step.
- **When to use:** Object with many optional fields; avoid telescoping constructors.
- **SD Application:** SQL query builder, HTTP request builder, complex config objects.

```
new QueryBuilder()
   .select("*")
   .from("users")
   .where("age > 18")
   .limit(10)
   .build()
```

Prototype

- **Problem:** Clone an existing object instead of creating from scratch.
- **When to use:** Object creation is expensive; create copies of a template.
- **SD Application:** Template documents, pre-configured server configurations.

STRUCTURAL PATTERNS

Adapter

- **Problem:** Make incompatible interfaces work together.
- **When to use:** Integrating third-party library with existing code.
- **SD Application:** Wrapping legacy payment API with new interface; DB driver adapter.

Decorator

- **Problem:** Add behavior to objects dynamically without altering their class.
- **When to use:** Adding features to objects at runtime (logging, caching, compression).
- **SD Application:** HTTP middleware (auth → rate limit → logging → handler); stream compression.

```
LoggingDecorator(CachingDecorator(DatabaseRepository()))
```

Facade

- **Problem:** Provide a simplified interface to a complex subsystem.
- **When to use:** Hide complexity; simplify API for clients.
- **SD Application:** Order service facade hiding inventory + payment + shipping services.

Proxy

- **Problem:** Provide a surrogate/placeholder to control access to an object.
- **Types:** Virtual proxy (lazy init), protection proxy (access control), remote proxy (remote object).
- **SD Application:** Caching proxy (cache DB results), API rate-limiting proxy, ORM lazy loading.

Composite

- **Problem:** Treat individual objects and compositions of objects uniformly.
- **When to use:** Tree structures, file systems, UI components.
- **SD Application:** File system (File and Folder both implement FileSystemItem), UI component tree.

BEHAVIORAL PATTERNS

Strategy

- **Problem:** Define a family of algorithms; make them interchangeable.
- **When to use:** Sorting, routing, pricing, payment — behavior varies by context.
- **SD Application:** Payment strategy (Credit Card, UPI, Wallet), sorting strategy, compression strategy.

```
PaymentProcessor(strategy=UPIStrategy)
PaymentProcessor(strategy=CardStrategy)
```

Observer

- **Problem:** When one object changes state, all dependents are notified automatically.
- **When to use:** Event systems, notifications, reactive UI.
- **SD Application:** Notification system (order placed → notify user, update inventory, send email), event-driven architecture.

Command

- **Problem:** Encapsulate a request as an object.
- **When to use:** Undo/redo, request queuing, logging operations.
- **SD Application:** Task queue (each task is a Command), undo in editors.

State

- **Problem:** Object's behavior changes based on its internal state.
- **When to use:** State machines — order status, vending machine states, traffic light.
- **SD Application:** Order (Placed → Confirmed → Shipped → Delivered → Cancelled), ATM states.

Chain of Responsibility

- **Problem:** Pass request through a chain of handlers; each decides to handle or pass.
- **When to use:** Middleware pipelines, event handling, approval workflows.
- **SD Application:** API middleware chain (Auth → Rate Limit → Logging → Handler).

Template Method

- **Problem:** Define skeleton of algorithm in base class; subclasses override steps.
- **When to use:** Algorithms with same structure but different steps.
- **SD Application:** Data import pipeline (parse → validate → transform → save), report generation.

---

18. SYSTEM DESIGN / ARCHITECTURE PATTERNS

Monolithic Architecture

- Single codebase and deployment unit.
- **Use:** Small teams, early stage, fast development.
- **Avoid:** When teams are large, services need independent scaling, or deployments are too risky.
- **Trade-off:** Simple to start; hard to scale specific parts; one bug can crash everything.

Microservices

- System split into small, independently deployable services.

- **Use:** Large teams, distinct bounded domains, independent scaling needs.
- **Avoid:** Small teams/early stage (operational overhead is high).
- **Trade-off:** Independent scaling + deployment vs distributed system complexity.

Event-Driven Architecture

- Services communicate via events (Kafka, RabbitMQ) instead of direct calls.
- **Use:** Decoupled services, async processing, audit logs.
- **Trade-off:** Hard to debug; eventual consistency; requires message broker.

CQRS (Command Query Responsibility Segregation)

- Separate read model from write model.
- Writes go to command store (optimized for writes); reads from query store (optimized for reads).
- **Use:** Read-heavy systems, complex domain models, analytics.
- **Trade-off:** Added complexity; write and read stores must stay in sync.

Event Sourcing

- Store all changes as a sequence of events, not current state.
- Replay events to reconstruct state.
- **Use:** Audit trails, financial systems, undo functionality.
- **Trade-off:** Complex queries; large event store; replay can be slow.

Saga Pattern

- Distributed transaction via sequence of local transactions + compensating transactions on failure.
- **Choreography:** Services emit events; each reacts to previous.
- **Orchestration:** Central saga orchestrator calls services in order.

- **Use:** Multi-service transactions (order → payment → inventory → shipping).
- **Trade-off:** Complex rollback logic; eventual consistency.

Circuit Breaker

- Prevent cascading failure by stopping calls to a failing service.
- **States:** Closed (normal) → Open (stop calls) → Half-Open (test recovery).
- **Use:** Any microservice calling external service.
- **Trade-off:** May reject legitimate requests during open state; tuning thresholds is hard.

API Gateway

- Single entry point for all clients; routes to microservices.
- Handles: Auth, rate limiting, SSL termination, logging, protocol translation.
- **Trade-off:** Becomes bottleneck if not scaled; adds latency.

Backend-for-Frontend (BFF)

- Separate API gateway/service for each client type (web, mobile, IoT).
- **Use:** When web and mobile have very different data needs.
- **Trade-off:** Duplicate logic; more services to maintain.

Strangler Pattern

- Gradually migrate monolith to microservices by replacing pieces one by one.
- Route traffic through a facade; migrate service by service.
- **Use:** Legacy system modernization without big-bang rewrite.

---

19. COMMON TRADE-OFFS

Decision	Option A	Option B	When to Choose A	When to Choose B
Database	SQL	NoSQL	Transactions, complex queries, relations	High write throughput, flexible schema, massive scale
Consistency	Strong	Eventual	Banking, inventory, payments	Social feeds, likes, analytics
Schema	Normalized	Denormalized	Data integrity, minimal storage	Read performance, fewer joins
Scaling	Vertical	Horizontal	Simple, small scale	Large scale, high availability
Communication	Synchronous	Asynchronous	Immediate feedback needed	Decoupling, resilience, heavy processing
Transport	TCP	UDP	Reliability required	Low latency, loss acceptable
API	REST	WebSocket	Standard CRUD	Real-time bidirectional

Replication	Read Replicas	Sharding	Read-heavy, single write point	Write-heavy, data doesn't fit on one machine
Architecture	Monolith	Microservices	Small team, early stage	Large org, independent scaling
Queue	Kafka	RabbitMQ	High throughput, log replay, event sourcing	Flexible routing, task queues, RPC
Availability vs Consistency	AP (available)	CP (consistent)	Social media, streaming	Banking, inventory

---

PART B — SCENARIO-BASED INTERVIEW QUESTIONS

SECTION A — DATABASE-CENTRIC SCENARIOS

Question 1 — Design a URL Shortener

What the Interviewer Is Testing

- Hash function design.
- Database selection for high reads.
- Caching strategy.
- Collision handling.
- Analytics collection.

1. Clarify Requirements

Functional Requirements

- User submits a long URL → gets a short URL.
- User visits short URL → redirected to long URL.
- Optional: custom aliases, expiry, analytics (clicks).

Non-Functional Requirements

- Availability: 99.99%.
- Latency: redirect < 50ms.
- Scale: 100M URLs created, 10B redirects/day.
- Short URLs must be unique and not predictable.

2. Assumptions & Scale

```
Writes (URL creation): 100M URLs → ~1,157 writes/sec
Reads (redirects): 10B/day → ~115,740 reads/sec
Read/Write ratio: ~100:1 (heavily read)
URL record size: ~1 KB
Storage: 100M × 1KB = ~100 GB
```

3. Core Entities

- **URL:** short_code, long_url, user_id, created_at, expires_at, click_count

4. Database Decision

Choose: PostgreSQL for URL mappings + Redis for redirect cache.

Why:

- URL data is relational and structured → SQL works perfectly.
- Short code → long URL is a key-value lookup → cache in Redis for 99% of reads.
- Write volume (1,157/sec) is easily handled by PostgreSQL.
- 100 GB fits comfortably on a single powerful PostgreSQL instance with read replicas.

Schema:

```
urls table:
 id           BIGSERIAL PRIMARY KEY
 short_code   VARCHAR(8) UNIQUE NOT NULL
 long_url     TEXT NOT NULL
 user_id      BIGINT
 created_at   TIMESTAMP DEFAULT NOW()
 expires_at   TIMESTAMP
 click_count  BIGINT DEFAULT 0
```

Indexes:

- `unique index on short_code` → O(1) lookup.
- `index on user_id` → for user's URL history.
- `index on expires_at` → for cleanup job.

Scaling strategy:

- Read replicas for high read traffic (100:1 ratio).
- Redis cache with TTL = 24h for hot short codes → eliminates DB reads for popular URLs.

5. API Design

```
POST /shorten
 Body: { long_url, custom_alias (optional), expires_in (optional) }
 Response: { short_url }

GET /{short_code}
 Response: 301/302 redirect to long_url

GET /api/analytics/{short_code}
 Response: { click_count, referrers, geo_data }
```

6. High-Level Architecture

```
Client
  ↓
Load Balancer
  ↓
API Servers (stateless, horizontally scalable)
  ↓
Redis Cache (short_code → long_url, TTL=24h)
  ↓ (cache miss)
PostgreSQL (primary + read replicas)
  ↓ (async)
Analytics Queue (Kafka) → Analytics Consumer → ClickHouse/Analytics DB
```

7. How It Works

1. **Shorten:** API server generates short code (Base62 on unique ID from DB sequence). Stores in PostgreSQL. Optionally pre-populates Redis.
2. **Redirect:** Client hits `/{short_code}` → API checks Redis → HIT: returns 301 redirect. MISS: queries PostgreSQL → stores in Redis → returns 301 redirect.
3. **Click analytics:** Redirect triggers async Kafka event → consumer aggregates click data.

8. Networking Decisions

- **Protocol:** HTTPS (TLS 1.3).
- **HTTP/2:** For multiplexed requests from browser.
- **Redirect:** 301 (permanent, cached by browser) vs 302 (temporary, not cached — use 302 for analytics accuracy).
- **CDN:** Not needed for redirect (dynamic per short code); useful for analytics dashboard assets.
- **Load balancing:** Round Robin for stateless API servers.

9. Caching

- **What is cached:** short_code → long_url mapping.
- **Cache strategy:** Cache-aside.

- **TTL:** 24 hours (or match URL expiry).
- **Invalidation:** Delete from cache on URL expiry or deletion.
- **Hot keys:** Popular short codes → local in-process cache on API server.

10. Async Processing

- **Queue:** Kafka topic `url_clicks`.
- **Producer:** API server publishes click event on redirect.
- **Consumer:** Analytics service aggregates click data into ClickHouse.
- **Why async:** Click counting doesn't need to block the redirect response.

11. Scalability

- **Application:** Stateless API servers — add more behind load balancer.
- **Database:** Read replicas for reads; write to primary only.
- **Cache:** Redis Cluster as traffic grows; local cache for extreme hot keys.
- **Short code generation:** Use database sequence or distributed ID generator (Snowflake) to avoid collision.

12. Reliability & Failure Handling

- **Server failure:** Load balancer health check routes away from failed servers.
- **Database failure:** Read replicas serve reads; auto-failover to replica promotes it to primary.
- **Cache failure:** Graceful degradation — fall back to database.
- **Retry:** No retry on redirect (user clicks again); analytics events retried via Kafka.

13. Security

- **Authentication:** JWT for URL creation (prevent anonymous abuse).
- **Authorization:** Users can only delete their own URLs.
- **Rate limiting:** 10 shortens per minute per IP.
- **Data protection:** Validate long URL (reject malware/phishing domains via blocklist).

- **Short code:** Base62 random — not sequential (prevent enumeration).

14. Bottlenecks

- Redis becomes bottleneck at extreme scale → Redis Cluster + local L1 cache.
- PostgreSQL write bottleneck for URL creation → unlikely at 1,157 writes/sec.
- Short code generation → use distributed ID (Snowflake) to avoid DB roundtrip.

15. Trade-offs

Decision	Why	Trade-off
PostgreSQL over NoSQL	Structured data, ACID, simple queries	NoSQL would scale writes better at extreme scale
301 vs 302 redirect	302 forces every redirect through server	Browser caches 301 → lose click analytics
Redis cache	Eliminate 99% DB reads	Cache invalidation complexity on URL update/delete
Base62 encoding	URL-safe, compact (8 chars → 62^8 = 218T unique)	Slightly longer than Base64

16. Common Interview Follow-Ups

Follow-up 1

Question: How do you generate unique short codes without collision?

Answer: Use a **database auto-increment sequence** as the unique ID; encode it in **Base62** (a-z, A-Z, 0-9). Since IDs are globally unique by definition, no collision is possible. Alternatively, use a **Snowflake ID** generator for distributed systems.

Follow-up 2

Question: What if a URL is viral and receives 1 million redirects per second?

Answer: Add a **local in-process cache** on each API server for the top N hot URLs. This avoids Redis entirely for the hottest short codes. Combine with horizontal scaling of API servers and Redis Cluster.

Follow-up 3

Question: How do you handle URL expiry?

Answer: Store `expires_at` in the DB. On redirect, check expiry. Run a **background cron job** to delete expired URLs and evict from cache. Use Redis TTL to auto-expire cache entries.

Final Interview Answer

> "For a URL shortener, the core challenge is a massively read-heavy system — about 100:1 read to write. My approach: generate short codes using Base62 encoding of a database sequence ID, store mappings in PostgreSQL for durability and ACID guarantees, and put a Redis cache in front of the DB for all redirects with a 24-hour TTL. This makes 99%+ of redirects pure cache hits at sub-millisecond latency. For analytics, I'd use Kafka to async-collect click events without blocking the redirect. The architecture is simple, stateless API servers behind a load balancer, read replicas for database scaling, and Redis Cluster as traffic grows. I'd use 302 redirects over 301 to avoid browser caching so analytics remain accurate."

Question 2 — Design a Rate Limiter

What the Interviewer Is Testing

- Understanding of rate limiting algorithms.
- Distributed consistency challenges.
- Redis usage for shared state.
- Handling edge cases (multiple servers, clock skew).

1. Clarify Requirements

Functional Requirements

- Limit each user/IP to N requests per time window.
- Return 429 Too Many Requests when limit exceeded.
- Support different limits for different API endpoints.

Non-Functional Requirements

- Latency: rate limiter check < 5ms.
- Availability: must not block all traffic if rate limiter fails.
- Accuracy: reasonable (not 100% exact at extreme scale).
- Distributed: must work across multiple API server instances.

2. Assumptions & Scale

```
Users: 10M DAU
Requests/sec: 100,000 RPS
Limit: 100 req/min per user
Rate limiter check must be < 5ms per request
```

3. Core Entities

- **RateLimitRecord:** user_id/IP, window_start, request_count

4. Database Decision

Choose: Redis (not a traditional DB).

Why:

- Rate limiting requires **sub-millisecond shared state** across all API servers.
- Redis is in-memory, atomic operations (INCR, EXPIRE), and supports Lua scripts for atomic check-and-increment.
- No disk persistence needed for rate limit counters.
- Redis TTL naturally handles window expiry.

Data Model:

```
Redis Key: rate_limit:{user_id}:{window}
Value: request_count (integer)
TTL: window_duration (e.g., 60 seconds)
```

Scaling strategy:

- Redis Cluster for horizontal scaling.
- Each key sharded across nodes by consistent hashing.

5. API Design

```
Rate limiter is middleware — not an external API.
Every request to API:
 → Rate limiter checks Redis
 → If count < limit: increment + allow
 → If count >= limit: return 429 with Retry-After header
```

6. High-Level Architecture

```
Client
  ↓
Load Balancer
  ↓
API Gateway / API Server
  ↓ (every request)
Rate Limiter Middleware
  ↓
Redis Cluster (rate limit counters)
  ↓ (allowed)
Backend Service
```

7. How It Works — Sliding Window Counter Algorithm

```
1. Request arrives for user_id=123
2. Redis key = "rl:123:{current_minute}"
```

```
3. Atomically: INCR key → returns new count
4. If new_count == 1: SET EXPIRE to 60 seconds
5. If new_count > limit (100): return 429
6. Else: allow request to proceed
```

8. Rate Limiting Algorithms

Algorithm	Description	Pros	Cons
Fixed Window	Count requests per fixed time window	Simple, fast	Burst at window boundary (2× limit possible)
Sliding Window Log	Store timestamp of each request	Accurate	High memory usage
Sliding Window Counter	Weighted count using current + previous window	Accurate, memory efficient	Slight approximation
Token Bucket	Tokens replenished at fixed rate; request consumes token	Allows bursts	More complex
Leaky Bucket	Requests queued; processed at fixed rate	Smooth output	Queue delay

> **Recommended for interviews:** Sliding Window Counter (best balance of accuracy and simplicity).

9. Caching

- Redis IS the "cache" here — no additional layer needed.

10. Async Processing

- Not required for this design.

11. Scalability

- **Redis Cluster:** Shard rate limit keys across nodes.
- **Local fallback:** If Redis is unreachable, apply a conservative local in-memory limit as fallback.

12. Reliability & Failure Handling

- **Redis failure:** Fail open (allow traffic) to avoid service outage. Alert ops team. Apply in-process fallback limit.
- **Network latency:** Rate limiter check must have a short timeout (< 10ms); on timeout → fail open.

13. Security

- Rate limit by user ID (authenticated) AND by IP (unauthenticated).
- Different limits for different endpoints (e.g., login: 5/min; search: 100/min).
- Return `Retry-After` header so well-behaved clients back off.

14. Trade-offs

Decision	Why	Trade-off
Redis over DB	Sub-ms latency, atomic INCR	Not durable; loses counts on Redis restart (acceptable)
Fail open on Redis failure	Availability over rate limiting	Brief period of unlimited requests
Sliding window counter	Memory efficient, accurate	Slight over/under counting at window boundary

16. Common Interview Follow-Ups

Follow-up 1

Question: How do you handle multiple API servers without Redis — purely local rate limiting?

Answer: You can't accurately rate limit distributed traffic with only local state. Local rate limiting would allow each server to accept N requests independently, so a user could send N×servers total requests. Redis shared state is necessary for accurate distributed rate limiting.

Follow-up 2

Question: How do you rate limit at the user tier level (free vs premium)?

Answer: Store the limit alongside the rate limit key or look up the user's tier from a cache on first request. Apply different `limit` values based on tier. Cache the user's tier in Redis or local memory for a short TTL.

Final Interview Answer

> "A rate limiter needs shared state across all API servers, so Redis is the natural choice — it's in-memory, has atomic operations (INCR), and TTL-based expiry handles window cleanup automatically. I'd implement a sliding window counter: for each request, increment a Redis key scoped to user+time_window. If the count exceeds the limit, return 429 with a Retry-After header. The key insight is to use Redis's INCR + EXPIRE atomically via a Lua script to avoid race conditions. For fault tolerance, I'd fail open if Redis is unreachable so we don't accidentally block all traffic."

Question 3 — Design Twitter/X (News Feed)

What the Interviewer Is Testing

- Fan-out architecture for news feed.
- Read vs write amplification trade-off.
- Caching feeds.
- Database for social graph.
- Handling celebrities (hot users).

1. Clarify Requirements

Functional Requirements

- User creates a tweet.
- User follows/unfollows other users.
- User sees home timeline (tweets from people they follow, reverse chronological).
- User sees a profile's tweets.
- Like, retweet (simplified).

Non-Functional Requirements

- Availability: 99.99%.
- Timeline load latency: < 200ms.
- Scale: 300M DAU, 500M tweets/day.
- Read-heavy: users check timeline far more often than they tweet.

2. Assumptions & Scale

```
DAU = 300M users
Tweets/day = 500M → ~5,800 writes/sec
Timeline reads/day = 10B+ → ~115,000 reads/sec
Average follows per user = 200
Read/Write ratio = ~20:1 (heavily read)
Tweet size = ~500 bytes
Storage/day = 500M × 500B = 250 GB/day
```

3. Core Entities

- **User:** user_id, username, bio, follower_count, following_count
- **Tweet:** tweet_id, user_id, content, created_at, like_count, retweet_count
- **Follow:** follower_id, followee_id, created_at
- **Feed:** user_id → [tweet_ids] (materialized/precomputed)

4. Database Decision

Multiple databases for different needs:

Data	Database	Why
User profiles	PostgreSQL	Structured, relational, ACID
Tweets	Cassandra	High write throughput, time-series, wide-column
Social graph (follows)	PostgreSQL or Redis	Follow/unfollow queries; graph traversal
News feed	Redis (sorted set)	Pre-built feed per user; sorted by timestamp
Media (images/video)	S3 + CDN	Binary objects

Cassandra for Tweets: - Partition key: `user_id`, Clustering key: `tweet_id DESC` (most recent first). - Allows efficient "get tweets for user X" queries. - Handles 5,800 writes/sec easily.

Redis for Feed: - Sorted set per user: `feed:{user_id}` → `{tweet_id: timestamp}`. - Get timeline = `ZREVRANGE feed:123 0 50`. - TTL to evict feeds for inactive users.

5. API Design

```
POST /tweets                          → Create tweet
DELETE /tweets/{tweet_id}             → Delete tweet
GET  /users/{user_id}/timeline        → Get profile tweets
GET  /home/timeline                   → Get home feed
POST /users/{user_id}/follow          → Follow user
DELETE /users/{user_id}/follow        → Unfollow user
POST /tweets/{tweet_id}/like          → Like tweet
```

6. High-Level Architecture

```
Client
  ↓
Load Balancer
  ↓
API Gateway (auth, rate limiting)
  ↓
████████████████████████████████████████
█ Tweet Service █ User Service █ Feed Service █
████████████████████████████████████████
  ↓                              ↓
Cassandra (tweets)          Redis (prebuilt feeds)
PostgreSQL (users, follows)
S3 + CDN (media)
  ↓
Kafka → Feed Fanout Service → Redis (write fan-out)
```

7. How It Works — Fan-Out on Write

When user A tweets: 1. Tweet saved to Cassandra. 2. Kafka event: `{tweet_id, user_id, timestamp}`. 3. **Fan-out worker** reads event from Kafka. 4. Fetches all followers of user A (from PostgreSQL). 5. For each follower, inserts tweet_id into their Redis sorted set feed.

When user reads timeline: 1. `GET /home/timeline` → Feed Service. 2. Fetch tweet IDs from Redis sorted set (`ZREVRANGE`). 3. Batch fetch full tweet data from Cassandra. 4. Return assembled timeline.

Trade-off: Fan-out on Write vs Fan-out on Read

Approach	Write	Read	Best For
Fan-out on Write	Slow (write to all followers)	Fast (prebuilt feed)	Most users (< 10K followers)

Fan-out on Read	Fast (just write tweet)	Slow (compute at read time)	Celebrities with millions of followers
Hybrid	Fan-out for normal users; pull for celebrities	Mixed	Production Twitter

Hybrid approach: For users with > 1M followers (celebrities), don't fan-out. Instead, fetch and inject their tweets at read time.

8. Networking Decisions

- **Protocol:** HTTPS.
- **HTTP/2:** Multiplexed for mobile clients with multiple requests.
- **CDN:** For profile images, video thumbnails.
- **WebSocket/SSE:** For real-time tweet notifications (optional; polling often sufficient for timeline).
- **Load balancing:** L7, route by user_id for sticky feed service if needed.

9. Caching

- **User profiles:** Redis, TTL = 1 hour.
- **Home feed:** Redis sorted set per user (precomputed, fan-out on write).
- **Tweet data:** Redis hash for hot tweets, TTL = 15 min.
- **Follow list:** Redis set for celebrity follower lists (avoid DB reads for fan-out).
- **Invalidation:** Delete tweet from feed sorted set + Cassandra on delete; update on like/retweet count.

10. Async Processing

- **Queue:** Kafka topic `tweet_created`.
- **Producer:** Tweet Service publishes tweet event.
- **Consumer:** Fan-out Service reads and pushes to follower feeds in Redis.

- **Why async:** Fan-out to 10,000 followers synchronously would time out the tweet request.

11. Scalability

- **Cassandra:** Natively horizontally scalable; add nodes.
- **Redis:** Redis Cluster; shard feeds by user_id.
- **Fan-out workers:** Scale consumer group horizontally in Kafka.
- **Timeline reads:** Stateless Feed Service scaled horizontally.

12. Reliability & Failure Handling

- **Cassandra node failure:** Replication factor 3; quorum writes/reads; auto-recovery.
- **Redis failure:** Fall back to computing feed from Cassandra (slower but correct); Redis is rebuildable.
- **Kafka failure:** Replication across brokers; consumers retry from last offset.
- **Fan-out lag:** Tweets may appear slightly delayed in follower feeds — acceptable eventual consistency.

13. Security

- **Authentication:** JWT for all API calls.
- **Private accounts:** Check follow status before allowing timeline read.
- **Rate limiting:** 300 tweets/day per user; 100 timeline reads/min.
- **Content moderation:** Async ML pipeline on Kafka for spam/inappropriate content detection.

14. Trade-offs

Decision	Why	Trade-off
Fan-out on Write	Sub-100ms timeline reads	High write amplification; celebrity problem

Cassandra for tweets	High write throughput	Eventual consistency; limited query flexibility
Redis sorted set for feed	O(log n) insert/read; natural time ordering	Memory cost; stale if not cleaned up
Hybrid fan-out	Solves celebrity problem	More complex code path

16. Common Interview Follow-Ups

Follow-up 1

Question: How do you handle a celebrity with 50 million followers posting a tweet?

Answer: Do not fan-out on write for celebrities. Flag users with > 1M followers as "celebrities." When a regular user reads their timeline, the Feed Service merges their prebuilt feed (from Redis) with a live query for celebrity tweets (from Cassandra, cached separately). This avoids writing to 50M Redis sorted sets.

Follow-up 2

Question: What happens if the fan-out service is slow and tweets appear late in follower feeds?

Answer: This is acceptable eventual consistency. A user sees the tweet a few seconds late — not a critical issue. Use Kafka consumer lag metrics to monitor. Scale fan-out workers to keep lag below 5 seconds.

Follow-up 3

Question: How do you implement trending topics?

Answer: Use a sliding window counter in Redis. For each tweet, extract hashtags and increment a sorted set `trending:{window}` with hashtag as key and count as score. ZREVRANGE gives top N trending topics. Aggregate across multiple servers using a dedicated aggregation service.

Final Interview Answer

> "Designing Twitter's feed is fundamentally a fan-out problem. My approach: store tweets in Cassandra for high write throughput — it handles 6,000+ writes/sec natively

> with wide-column storage partitioned by user. For the home timeline, I precompute feeds using fan-out on write — when a tweet is posted, a Kafka consumer pushes the tweet ID to each follower's Redis sorted set. Timeline reads become a simple Redis range query plus a batch fetch from Cassandra — that's sub-100ms. The catch is celebrities: fan-out for 50M followers would overwhelm the system, so I use a hybrid approach — pre-build feeds only for normal users and inject celebrity tweets at read time. The key database decision is using multiple stores for different needs: Cassandra for tweets, Redis for feeds, PostgreSQL for the social graph."

Question 4 — Design WhatsApp / Chat System

What the Interviewer Is Testing

- Real-time messaging architecture.
- WebSocket vs polling.
- Message storage and ordering.
- Delivery status (sent, delivered, read).
- Offline message delivery.

1. Clarify Requirements

Functional Requirements

- One-on-one messaging.
- Group messaging (up to 256 members).
- Message delivery status: Sent → Delivered → Read.
- Push notifications for offline users.
- Message history.

Non-Functional Requirements

- Latency: message delivered < 100ms (online users).
- Availability: 99.99%.
- Scale: 2B users, 100B messages/day.

- Messages must not be lost.
- Messages ordered within a conversation.

2. Assumptions & Scale

```
Users: 2B total, 500M DAU
Messages/day: 100B → ~1.15M messages/sec
Avg message size: 1KB (text + metadata)
Storage/day: 100B × 1KB = 100 TB/day
Read/Write: roughly equal (each message read by recipient)
```

3. Core Entities

- **User:** user_id, name, phone, status
- **Message:** message_id, conversation_id, sender_id, content, type, sent_at, status
- **Conversation:** conversation_id, type (1-1/group), participant_ids, last_message_at
- **MessageStatus:** message_id, user_id, status (sent/delivered/read), timestamp

4. Database Decision

Choose: Cassandra for messages + PostgreSQL for user/conversation metadata + Redis for online presence.

Why Cassandra for messages: - 1.15M messages/sec write throughput — Cassandra is designed for this. - Messages are time-ordered per conversation → natural partition by conversation_id + time clustering key. - Append-only write pattern (messages never updated, only status changes). - Horizontally scalable.

Schema:

```
messages table (Cassandra):
 conversation_id  UUID   PARTITION KEY
 message_id       TIMEUUID  CLUSTERING KEY (DESC)
 sender_id        UUID
 content          TEXT
 type             TEXT  (text/image/video)
 status           TEXT  (sent/delivered/read)
```

```
 sent_at           TIMESTAMP

users table (PostgreSQL):
 user_id           UUID PRIMARY KEY
 phone             VARCHAR UNIQUE
 name              VARCHAR
 created_at        TIMESTAMP
 last_seen         TIMESTAMP

conversations table (PostgreSQL):
 conversation_id   UUID PRIMARY KEY
 type              VARCHAR  (direct/group)
 created_at        TIMESTAMP
 last_message_at   TIMESTAMP
```

Indexes:

- Cassandra: partition by `conversation_id`; cluster by `message_id DESC` for time-ordered retrieval.
- PostgreSQL: `users.phone` for login lookup; `conversations.last_message_at` for inbox ordering.

5. API Design

```
WebSocket connection:
 ws://chat.server/{user_id}  (persistent connection while online)

REST APIs:
 POST /messages                → Send message
 GET  /conversations/{id}/messages?before={msg_id}&limit=50  → Load histor
 POST /conversations           → Create new conversation
 PUT  /messages/{id}/status    → Update delivery status
 GET  /conversations           → Get user's inbox
```

6. High-Level Architecture

```
Client (Mobile/Web)
  ↓ WebSocket
Load Balancer (L7, sticky sessions by user_id)
  ↓
Chat Servers (WebSocket connections)
  ↓
Message Queue (Kafka) → Message Service → Cassandra
  ↓
Presence Service (Redis: user → server mapping)
  ↓
Notification Service → APNs/FCM (for offline users)
```

7. How It Works

Sending a message (online recipient): 1. User A's client sends message via WebSocket to Chat Server A. 2. Chat Server A writes message to Cassandra (async via Kafka). 3. Chat Server A looks up Recipient B's Chat Server in Redis (`presence:{user_b} → server_id`). 4. Routes message to Chat Server B. 5. Chat Server B delivers to User B's WebSocket. 6. User B's client sends delivery ACK → status updated to "Delivered."

Sending a message (offline recipient): 1. Steps 1–3 same. 2. Redis shows User B is offline. 3. Notification Service sends push notification via APNs/FCM. 4. When User B comes online and opens app → fetches unread messages from Cassandra. 5. Status updated to "Delivered" then "Read."

8. Networking Decisions

- **Protocol:** WebSocket for real-time bidirectional messaging.
- **Why WebSocket over polling:** Persistent connection; no overhead of repeated HTTP requests; server can push messages immediately.
- **Fallback:** Long-polling for clients that don't support WebSocket.
- **Connection management:** Each Chat Server maintains a map of `user_id → WebSocket connection`.
- **Load balancing:** IP Hash or Cookie-based for WebSocket stickiness (same user always to same server).
- **Heartbeat:** Client sends ping every 30s; server disconnects on timeout.

9. Caching

- **User online status:** Redis key `online:{user_id}` with TTL = 60s (refreshed by heartbeat).
- **User → Chat Server mapping:** Redis `user_server:{user_id} → server_id`.
- **Recent messages:** Redis list per conversation for last 20 messages (avoids Cassandra reads for active chats).
- **User profiles:** Redis hash, TTL = 1 hour.

10. Async Processing

- **Queue:** Kafka topic `messages`.
- **Producer:** Chat Server publishes message after WebSocket receipt.
- **Consumer:** Message Service persists to Cassandra; Notification Service for offline users; Analytics.
- **Why async:** Persistence doesn't need to block message delivery to online recipient.

11. Scalability

- **Chat Servers:** Stateless per message but stateful WebSocket connections. Scale horizontally; use Redis for cross-server routing.
- **Cassandra:** Add nodes; replication factor 3; scales to petabytes.
- **Kafka:** Partition by conversation_id for ordering; scale consumer groups.
- **Presence Service:** Redis Cluster sharded by user_id.

12. Reliability & Failure Handling

- **Chat Server failure:** Client detects WebSocket disconnect → reconnects (auto-reconnect with exponential backoff) → load balancer routes to new server → fetch missed messages from Cassandra.
- **Cassandra failure:** Replication factor 3; quorum writes ensure durability.
- **Kafka failure:** Messages replicated across brokers; at-least-once delivery.
- **Duplicate messages:** Client uses message_id for deduplication on receive.
- **Offline tolerance:** Messages durably stored in Cassandra; user fetches on reconnect.

13. Security

- **Authentication:** JWT per WebSocket connection.
- **Encryption in transit:** TLS (WSS — WebSocket Secure).
- **End-to-end encryption:** Signal Protocol (WhatsApp uses this); keys never leave client devices.
- **Group permissions:** Only members can send/read group messages.
- **Rate limiting:** 100 messages/min per user.

14. Trade-offs

Decision	Why	Trade-off
WebSocket over REST	Real-time, low latency, server push	Stateful connections; load balancing complexity
Cassandra for messages	Massive write throughput, time-ordered by partition	Limited query patterns; no cross-conversation queries
Async persistence	Faster delivery	Message may appear delivered before persisted
Redis for presence	Sub-ms lookup of user's server	Redis failure = can't route messages (fallback: broadcast to all servers)

16. Common Interview Follow-Ups

Follow-up 1

Question: How do you handle message ordering in group chats?

Answer: Cassandra's TIMEUUID clustering key provides time-ordered storage per conversation. For strict ordering, use a server-side timestamp (not client — client clocks can be wrong). For concurrent messages in the same millisecond, TIMEUUID's UUID

component provides a total order.

Follow-up 2

Question: How do you scale to 1 billion concurrent WebSocket connections?

Answer: Horizontally scale Chat Servers. Each server can hold ~100K WebSocket connections. 1 billion connections needs ~10,000 servers. Use Redis cluster to maintain user → server mapping. Route messages between servers via direct HTTP/gRPC call or via Kafka.

Final Interview Answer

> "The core challenge in designing WhatsApp is real-time delivery with persistence. For real-time, I'd use WebSocket — persistent bidirectional connections managed by Chat Servers. When a message is sent, the Chat Server routes it to the recipient's server using a Redis lookup. For storage, Cassandra is the right choice: 1 million+ messages per second write throughput, with natural time-ordered partitioning by conversation. For offline users, the Chat Server triggers push notifications via APNs/FCM. Message persistence happens asynchronously via Kafka so it doesn't add latency to delivery. The key insight is separating concerns: real-time routing via WebSocket+Redis, durable storage via Cassandra, offline delivery via push notifications."

Question 5 — Design YouTube / Video Streaming

What the Interviewer Is Testing

- Video storage and streaming.
- CDN for global delivery.
- Video transcoding pipeline.
- Metadata vs binary storage separation.
- Read-heavy optimization.

1. Clarify Requirements

Functional Requirements

- Users upload videos.

- Users watch videos (streaming, adaptive bitrate).
- Users search for videos.
- Video recommendations (simplified).
- Comments, likes (simplified).

Non-Functional Requirements

- Upload availability: 99.9%.
- Streaming latency: < 2 seconds to start.
- Scale: 2B users, 500 hours of video uploaded/minute, 1B hours watched/day.
- Video quality: multiple resolutions (360p, 720p, 1080p, 4K).

2. Assumptions & Scale

```
Videos uploaded: 500 hrs/min = 30,000 hrs/hour
Avg video size (raw): 1 GB/hr → 30 TB raw uploads/hour
After compression (multiple resolutions): ~5× original → 150 TB/hour
Video watched: 1B hrs/day → ~41M hrs/hour
Concurrent viewers: millions at peak
```

3. Core Entities

- **Video:** video_id, title, description, user_id, duration, status, created_at, view_count
- **VideoFile:** video_id, resolution, format, storage_url, size
- **User:** user_id, name, channel_name, subscriber_count
- **Comment:** comment_id, video_id, user_id, content, created_at
- **Subscription:** subscriber_id, channel_id

4. Database Decision

Data	Database	Why

Video metadata	PostgreSQL	Structured, searchable, relational
Video files	S3 / GCS (Object Storage)	Designed for large binary blobs; cheap; durable
Video search	Elasticsearch	Full-text search on titles, descriptions, tags
Comments	Cassandra	High write volume, time-ordered per video
User sessions	Redis	Fast session lookups
View counts	Redis (increment) → async → PostgreSQL	Atomic increment; avoid DB write per view
Recommendations	Separate ML service with own store	Complex computation

Schema (PostgreSQL — videos):

```
videos:
 video_id     UUID PRIMARY KEY
 title        VARCHAR(200)
 description  TEXT
 user_id      UUID REFERENCES users(user_id)
 duration_sec INTEGER
 status       VARCHAR (processing/active/deleted)
 view_count   BIGINT DEFAULT 0
 created_at   TIMESTAMP
 tags         TEXT[]
 thumbnail_url TEXT

video_files:
```

```
video_id     UUID
resolution   VARCHAR  (360p/720p/1080p/4K)
codec        VARCHAR  (H.264/H.265)
storage_key  TEXT     (S3 key)
size_bytes   BIGINT
PRIMARY KEY (video_id, resolution)
```

5. API Design

```
POST /videos/upload                → Get signed S3 URL for upload
POST /videos/{id}/process          → Trigger transcoding (after upload)
GET  /videos/{id}                  → Get video metadata + stream URLs
GET  /videos/{id}/stream?res=720p → Streaming URL (signed CDN URL)
GET  /search?q={query}             → Search videos
POST /videos/{id}/comments         → Add comment
GET  /videos/{id}/comments         → Get comments (paginated)
POST /videos/{id}/like             → Like video
```

6. High-Level Architecture

```
Upload Flow:
Client → API → S3 (direct upload via signed URL)
                  ↓
            Upload Complete Event (S3 → Kafka)
                  ↓
        Transcoding Service (FFmpeg workers)
                  ↓
        S3 (360p, 720p, 1080p, 4K versions stored)
                  ↓
        Metadata DB updated (status = active)
                  ↓
        Elasticsearch indexed

Watch Flow:
Client → API → Metadata DB (PostgreSQL)
             → CDN (serves video segments)
```

```
                → Stream manifest (HLS/DASH playlist)
                → CDN edge delivers video chunks
```

7. How It Works

Upload: 1. Client requests signed S3 URL from API. 2. Client uploads video directly to S3 (bypasses application servers). 3. S3 upload triggers event → Kafka → Transcoding Service. 4. Transcoding workers use FFmpeg to create multiple resolutions (360p, 720p, 1080p). 5. Each transcoded file stored back in S3. 6. Metadata DB updated; Elasticsearch indexed for search.

Streaming: 1. Client requests video → API returns metadata + HLS manifest URL (from CDN). 2. HLS manifest lists available resolutions and segment URLs. 3. Client's video player fetches segments from CDN edge nodes (closest to user). 4. Adaptive Bitrate (ABR) — player monitors bandwidth; switches resolution dynamically. 5. View counted asynchronously via Kafka → Redis increment → periodic sync to PostgreSQL.

8. Networking Decisions

- **Protocol:** HTTPS for API; HLS (HTTP Live Streaming) over HTTPS for video.
- **CDN:** Absolutely critical — 1B hours watched/day; impossible without CDN. Cloudflare, Akamai, or own CDN.
- **Adaptive Bitrate:** HLS or DASH — player automatically picks best quality for current bandwidth.
- **TCP:** Video streaming over TCP (HTTP) for reliable delivery. UDP-based (WebRTC) for live streaming.
- **Connection:** HTTP/2 for multiplexed video segment requests.

9. Caching

- **Video metadata:** Redis, TTL = 10 min (title, description, view count).
- **Video segments:** CDN caches heavily; 90%+ cache hit rate for popular videos.
- **Search results:** Redis cache for popular queries, TTL = 5 min.
- **Signed URLs:** CDN pre-signs URLs; short TTL (4 hours) for DRM.
- **View count:** Redis INCR, async flush to PostgreSQL every minute.

10. Async Processing

- **Transcoding:** Kafka event triggers transcoding workers (horizontally scalable worker pool).
- **View counting:** Redis increment async → batch flush to PostgreSQL.
- **Notification:** Subscriber notification via Kafka → Notification Service → push/email.
- **Recommendations:** Kafka stream of user watch events → ML pipeline → Recommendation DB.

11. Scalability

- **Upload:** S3 handles unlimited concurrent uploads.
- **Transcoding:** Scale FFmpeg worker pool by adding more instances; queue-based (Kafka).
- **Streaming:** CDN handles 99% of traffic; scales globally.
- **Metadata DB:** Read replicas; cache popular videos.
- **Search:** Elasticsearch cluster, horizontally scaled.

12. Reliability & Failure Handling

- **Upload failure:** Client retries; S3 multipart upload for large files (resume on failure).
- **Transcoding failure:** Kafka retry with DLQ; re-queue failed jobs.
- **CDN failure:** Fallback to origin (S3); client retries automatically (HLS).
- **DB failure:** Read replicas; auto-failover.

13. Security

- **Authentication:** JWT for upload; signed URLs for streaming (cannot share).
- **DRM:** Digital Rights Management for premium content.
- **Content moderation:** Async ML pipeline post-upload for policy violations.
- **Rate limiting:** Upload: 5 videos/hour per user; API: 1000 req/min.

14. Trade-offs

Decision	Why	Trade-off
Client uploads directly to S3	Bypasses application servers; faster; cheaper	Cannot inspect content mid-upload
CDN for video delivery	Handles massive scale globally	Cache invalidation for updated/deleted videos
Async transcoding	Upload request returns immediately	User waits before video is watchable
Redis for view counts	Avoids DB write on every view	Slight inaccuracy; count flushed periodically

Final Interview Answer

> "YouTube has two distinct flows: upload and streaming. For uploads, clients upload directly to S3 via signed URLs, bypassing application servers. This triggers an async transcoding pipeline via Kafka, using FFmpeg workers to produce multiple resolutions. For streaming, the key is CDN — serving 1B hours of video without CDN is impossible. I'd use HLS for adaptive bitrate streaming: the player downloads a manifest listing video segments and automatically switches quality based on bandwidth. Video metadata lives in PostgreSQL; full-text search in Elasticsearch. The most important decisions are: object storage for binary data, CDN as the primary delivery mechanism, and async transcoding to keep upload latency low."

Question 6 — Design Uber / Cab Booking System

What the Interviewer Is Testing

- Location data management.
- Real-time matching algorithm.
- Geospatial queries.
- WebSocket for live tracking.

- Consistency in booking (no double booking).

1. Clarify Requirements

Functional Requirements

- Rider requests a ride.
- System matches with nearest available driver.
- Real-time driver location tracking.
- Fare estimation and payment.
- Trip history.

Non-Functional Requirements

- Availability: 99.99%.
- Matching latency: < 2 seconds.
- Location update frequency: every 5 seconds per driver.
- Scale: 1M concurrent drivers, 5M concurrent riders.

2. Assumptions & Scale

```
Active drivers: 1M (peak)
Active riders: 5M (peak)
Location updates: 1M drivers × 1 update/5sec = 200,000 location writes/sec
Ride requests: 10M rides/day → ~115 requests/sec
```

3. Core Entities

- **User:** user_id, name, email, phone, payment_method
- **Driver:** driver_id, user_id, vehicle_info, rating, current_location, status (available/busy/offline)
- **Trip:** trip_id, rider_id, driver_id, pickup, dropoff, status, fare, created_at
- **Location:** driver_id, latitude, longitude, updated_at

4. Database Decision

Data	Database	Why
User/Driver profiles	PostgreSQL	Relational, structured
Driver real-time location	Redis (Geo)	In-memory geo spatial; 200K writes/sec
Trip data	PostgreSQL	ACID transactions (booking must not double-assign)
Trip history	Cassandra	High volume, append-only, time-ordered
Geospatial queries	Redis GEO or PostGIS	Find nearest drivers

Redis GEO for driver locations:

```
GEOADD drivers:available {lng} {lat} {driver_id}
GEORADIUS drivers:available {rider_lng} {rider_lat} 5km ASC COUNT 10
```

Schema (PostgreSQL — trips):

```
trips:
 trip_id      UUID PRIMARY KEY
 rider_id     UUID REFERENCES users
 driver_id    UUID REFERENCES drivers
 pickup_lat   DECIMAL
 pickup_lng   DECIMAL
 dropoff_lat  DECIMAL
 dropoff_lng  DECIMAL
 status       VARCHAR  (requested/accepted/in_progress/completed/cancelled
```

```
fare         DECIMAL
requested_at TIMESTAMP
completed_at TIMESTAMP
```

Indexes: - `trips.rider_id` + `trips.driver_id` for history queries. - `trips.status` for finding active trips.

5. API Design

```
POST /rides/request           → Rider requests ride {pickup, dropoff}
GET  /rides/{id}/status       → Get ride status
POST /rides/{id}/accept       → Driver accepts
POST /rides/{id}/start        → Driver starts trip
POST /rides/{id}/complete     → Driver completes trip
WebSocket /drivers/location → Driver streams location
WebSocket /rides/{id}/track → Rider tracks driver location
POST /rides/{id}/rate         → Rate the trip
```

6. High-Level Architecture

```
Rider App                         Driver App
   ↓ REST                            ↓ WebSocket (location stream)
Load Balancer
   ↓
API Gateway (auth, rate limit)
   ↓
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Ride Service   ■ Location Service■ Payment   ■
■ (matching,trip)■ (driver positions■ Service   ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
   ↓               ↓                    ↓
PostgreSQL      Redis GEO           Payment Gateway
(trips, users)  (driver locations)
   ↓
Kafka → Trip events → Notification Service → APNs/FCM
```

7. How It Works

Ride Request: 1. Rider submits pickup/dropoff. 2. Ride Service estimates fare; creates trip record (status=requested). 3. Location Service queries Redis GEO for nearest available drivers within 5km. 4. Ride Service sends ride offer to top 3 drivers (via WebSocket/push). 5. First driver to accept: atomically set `driver_id` on trip + mark driver as busy (Redis transaction). 6. Both rider and driver notified via WebSocket.

Live Tracking: 1. Driver app sends GPS coordinates every 5 seconds via WebSocket. 2. Location Service writes to Redis GEO (in-memory, fast). 3. Rider app polls or subscribes to driver's location via WebSocket. 4. Location Service pushes driver coordinates to rider's WebSocket connection.

8. Networking Decisions

- **Driver location updates:** WebSocket (persistent connection; driver app streams location every 5s).
- **Rider tracking:** WebSocket (server pushes driver location to rider).
- **Ride matching API:** REST (request-response, no persistent connection needed).
- **CDN:** Not required for this design.
- **Load balancing:** IP Hash for WebSocket stickiness (driver → same server).

9. Caching

- **Driver locations:** Redis GEO is the cache (in-memory) — primary store for real-time locations.
- **Nearby drivers result:** Cache `GEORADIUS` result for 5 seconds (avoid re-querying for every ride request in same area).
- **Fare estimates:** Cache surge pricing multiplier per area, TTL = 1 min.
- **User/Driver profiles:** Redis hash, TTL = 30 min.

10. Async Processing

- **Queue:** Kafka topic `trip_events` (trip requested, accepted, completed).
- **Producer:** Ride Service publishes on every status change.
- **Consumer:** Notification Service sends push notifications; Analytics; Payment trigger on completion.

- **Why async:** Notification and analytics should not block the core trip flow.

11. Scalability

- **Location Service:** Stateless; Redis handles 200K writes/sec easily with cluster.
- **Ride Service:** Stateless; scale horizontally.
- **Redis GEO:** Partition by geographic region (shard by geo hash prefix).
- **WebSocket servers:** Scale horizontally; Redis pub/sub for cross-server driver location routing.

12. Reliability & Failure Handling

- **Redis failure:** Location data is ephemeral (drivers send updates every 5s); brief outage means slightly stale locations but self-heals quickly. Drivers reconnect and resume streaming.
- **Ride double-booking:** Use PostgreSQL `UPDATE trips SET driver_id=? WHERE trip_id=? AND driver_id IS NULL` — only one driver wins the CAS (Compare-and-Set).
- **Driver WebSocket disconnect:** Trip continues; driver reconnects with trip_id to resume.
- **Payment failure:** Retry payment async; mark trip as "payment_pending"; retry up to 3 times before flagging for manual review.

13. Security

- **Authentication:** JWT for all requests.
- **Driver verification:** KYC verified before allowing trip acceptance.
- **Payment:** PCI-DSS compliant payment service; never store raw card data.
- **Location privacy:** Rider sees only driver location during active trip; driver location not exposed after trip.

14. Trade-offs

Decision	Why	Trade-off

Redis GEO for locations	Sub-ms geospatial queries; 200K writes/sec	Ephemeral data; must accept some staleness
WebSocket for tracking	Real-time location push	Stateful connections; complex load balancing
PostgreSQL for trips	ACID prevents double booking	Lower write throughput (acceptable; 115 trips/sec)
Send to 3 drivers simultaneously	Reduces wait time	Multiple drivers must coordinate; need atomic assignment

Final Interview Answer

> "Uber is essentially a real-time matching and tracking system. The critical insight is separating driver location (extremely high write frequency) from trip management (needs ACID). For driver locations, I'd use Redis GEO — it handles 200K location updates per second in memory and supports geospatial queries to find nearby drivers. For trips, PostgreSQL with proper transaction handling prevents double-booking when multiple drivers accept simultaneously. Real-time tracking uses WebSocket — drivers stream GPS every 5 seconds; the server pushes those coordinates to the rider's WebSocket. The matching algorithm queries Redis GEO for drivers within 5km, offers the ride to the top 3 candidates, and uses an atomic DB update to assign the first accepting driver."

Question 7 — Design a Payment System

What the Interviewer Is Testing

- ACID transactions for money movement.
- Idempotency to prevent double charges.
- Distributed transaction challenges.
- Security and compliance.

- Retry handling without double charge.

1. Clarify Requirements

Functional Requirements

- User initiates payment.
- Money deducted from sender, credited to receiver.
- Payment status: Pending → Completed / Failed.
- Payment history.
- Support refunds.

Non-Functional Requirements

- Consistency: **Strong consistency** — money must not be lost or double-charged.
- Availability: 99.99%.
- Latency: payment confirmed < 3 seconds.
- Durability: every transaction persisted.
- Idempotency: retried payments must not charge twice.

2. Assumptions & Scale

```
Payments/day: 10M → ~115 payments/sec
Peak: 3× → ~350 payments/sec
Average payment size: $50
Total daily volume: $500M
Latency: < 3 seconds end-to-end
```

3. Core Entities

- **Account:** account_id, user_id, balance, currency, status
- **Payment:** payment_id, idempotency_key, from_account, to_account, amount, status, created_at
- **Transaction:** transaction_id, account_id, type (debit/credit), amount, balance_after, payment_id

4. Database Decision

Choose: PostgreSQL with strict ACID transactions. This is non-negotiable for financial data.

Why PostgreSQL: - Money movement MUST be atomic: debit sender + credit receiver in one transaction. - Strong consistency required — eventual consistency would risk inconsistent balances. - Foreign key integrity, constraints prevent invalid states. - 350 payments/sec is well within PostgreSQL's capability. - Proven at massive scale (Stripe, PayPal use PostgreSQL).

Schema:

```
accounts:
 account_id   UUID PRIMARY KEY
 user_id      UUID REFERENCES users
 balance      DECIMAL(15,2) NOT NULL CHECK (balance >= 0)
 currency     CHAR(3)
 status       VARCHAR (active/frozen/closed)
 version      INT DEFAULT 0  (optimistic locking)
 updated_at   TIMESTAMP

payments:
 payment_id       UUID PRIMARY KEY
 idempotency_key  VARCHAR(64) UNIQUE  ← prevents duplicate charge
 from_account_id  UUID REFERENCES accounts
 to_account_id    UUID REFERENCES accounts
 amount           DECIMAL(15,2)
 status           VARCHAR (pending/completed/failed/refunded)
 external_ref     VARCHAR  (bank reference)
 created_at       TIMESTAMP
 completed_at     TIMESTAMP

ledger:
 entry_id     UUID PRIMARY KEY
 account_id   UUID REFERENCES accounts
 payment_id   UUID REFERENCES payments
 type         VARCHAR (debit/credit)
 amount       DECIMAL(15,2)
 balance_after DECIMAL(15,2)
```

```
 created_at   TIMESTAMP
```

Indexes: - `payments.idempotency_key` (UNIQUE) → core for idempotency. - `payments.from_account_id` + `payments.created_at` → payment history. - `ledger.account_id` + `ledger.created_at` → account statement.

Scaling strategy: - Read replicas for balance checks and history queries. - Shard by account_id if needed (but keep both accounts of a transaction on same shard using consistent hashing — hard problem).

5. API Design

```
POST /payments
 Body: {
   idempotency_key: "uuid-unique-per-request",
   from_account_id: "...",
   to_account_id: "...",
   amount: 100.00,
   currency: "USD"
 }
 Response: { payment_id, status }

GET /payments/{payment_id}     → Get payment status
POST /payments/{payment_id}/refund → Initiate refund
GET /accounts/{id}/statement   → Get transaction history
```

6. High-Level Architecture

```
Client
  ↓
API Gateway (auth, rate limiting, TLS)
  ↓
Payment Service
  ↓ (idempotency check)
Idempotency Store (Redis + PostgreSQL)
  ↓ (valid new payment)
```

```
PostgreSQL Transaction:
 BEGIN;
 UPDATE accounts SET balance = balance - amount WHERE account_id = sender;
 UPDATE accounts SET balance = balance + amount WHERE account_id = receive
 INSERT INTO payments ...;
 INSERT INTO ledger (debit entry);
 INSERT INTO ledger (credit entry);
 COMMIT;
  ↓
Kafka (payment.completed) → Notification Service
  ↓
External Banking API (if cross-bank transfer)
```

7. Idempotency — Critical Pattern

```
Client sends payment with idempotency_key = "abc-123"
 ↓
Payment Service checks Redis: "idempotency:abc-123" exists?
 ↓
If EXISTS → return cached response (do NOT charge again)
If NOT EXISTS → process payment
             → store result in Redis (TTL = 24h)
             → return response
```

- **If client retries (network error):** Same idempotency_key → returns original response, not a second charge.
- **Idempotency key:** Client-generated UUID per payment attempt.

8. Handling Race Conditions

Optimistic Locking:

```
UPDATE accounts
SET balance = balance - 100, version = version + 1
WHERE account_id = 'abc' AND version = 5 AND balance >= 100;
```

- If 0 rows updated → someone else updated first → retry.

Pessimistic Locking:

```
SELECT * FROM accounts WHERE account_id = 'abc' FOR UPDATE;
-- lock held; proceed with debit
```

9. Async Processing

- **Queue:** Kafka topic `payment_events`.
- **Producer:** Payment Service publishes after successful DB commit.
- **Consumer:** Notification Service (receipt to user), Ledger Service (reconciliation), Fraud Detection.
- **Why async:** These can be decoupled from the payment critical path.

10. Scalability

- **Payment Service:** Stateless; scale horizontally.
- **Database:** Read replicas for queries; PostgreSQL handles 350 writes/sec on a single primary.
- **If scale exceeds single DB:** Shard by account_id (challenge: two-phase commit for cross-shard transactions → use Saga instead).

11. Reliability & Failure Handling

- **DB failure:** Synchronous replication; automatic failover; transaction either commits or rolls back fully.
- **Partial failure:** If service crashes after debit but before credit → DB transaction is atomic; rollback ensures both or neither.
- **External bank timeout:** Save payment as "pending"; retry external call with idempotency key; background reconciliation job.
- **Double charge prevention:** Idempotency key + DB UNIQUE constraint = two layers of protection.

12. Security

- **Authentication:** OAuth + JWT with short expiry.
- **Authorization:** Users can only initiate payments from their own accounts.
- **Encryption in transit:** TLS 1.3 everywhere.
- **Encryption at rest:** Database encrypted (AES-256); card data never stored (use tokenization).
- **PCI-DSS compliance:** External card processing via certified gateway (Stripe, Braintree).
- **Fraud detection:** Async ML scoring on Kafka stream; flag/block suspicious transactions.
- **Rate limiting:** 5 payments/min per user; 3 failed attempts → temporary block.

14. Trade-offs

Decision	Why	Trade-off
PostgreSQL (strong consistency)	Money cannot be lost or doubled	Lower throughput than NoSQL; single primary bottleneck
Idempotency via unique key	Prevent double charge on retry	Requires clients to generate and store idempotency keys
Synchronous debit+credit in one transaction	Atomicity guaranteed	Both accounts must be on same DB shard
Pessimistic locking	Prevents race conditions on same account	Reduced concurrency on hot accounts

Final Interview Answer

> "Payment systems demand strong consistency above all else — this rules out NoSQL. PostgreSQL with ACID transactions ensures that debit and credit happen atomically: either both succeed or neither does. The second most important design decision is

> idempotency: every payment request carries a client-generated idempotency key; if the same key is seen again, we return the original response without charging again. This prevents double charges on network retries. I'd use optimistic locking with a version column to handle concurrent payment attempts on the same account. For fraud detection and notifications, Kafka decouples these from the critical payment path. The hardest scaling challenge is cross-shard transactions if we shard the account table — in that case, I'd use the Saga pattern with compensating transactions rather than distributed 2PC."

Question 8 — Design Google Drive / Dropbox (File Storage)

What the Interviewer Is Testing

- Chunked file upload design.
- Sync across devices.
- Delta sync (only changed chunks).
- Object storage vs database.
- Conflict resolution.

1. Clarify Requirements

Functional Requirements

- Upload files (any size).
- Download files.
- Sync files across multiple devices.
- Share files with others.
- File versioning (restore older versions).

Non-Functional Requirements

- Availability: 99.99%.
- Durability: 99.999999999% (11 nines — S3 level).
- Scale: 500M users, 1B files, avg file 1 MB.

- Large file support: up to 50 GB.
- Sync latency: < 5 seconds for small files.

2. Assumptions & Scale

```
Users: 500M; 200M DAU
Files: 1B files × 1MB avg = 1 PB total storage
Uploads: 100M files/day → 1,157 uploads/sec
Downloads: 500M downloads/day → 5,787 downloads/sec
Sync events: 1B+ device events/day
```

3. Core Entities

- **User:** user_id, email, storage_used, storage_quota
- **File:** file_id, owner_id, name, size, mime_type, created_at, updated_at, is_deleted
- **FileVersion:** version_id, file_id, chunk_list, created_at, created_by
- **Chunk:** chunk_hash (SHA-256), size, storage_key (S3 key)
- **FileShare:** file_id, shared_with_user_id, permission (read/write)
- **DeviceSync:** device_id, user_id, last_sync_token

4. Database Decision

Data	Database	Why
File metadata	PostgreSQL	Relational, versioning, sharing
File content (binary)	S3 / GCS	Object storage designed for blobs
Chunk deduplication	PostgreSQL	chunk_hash lookup; content-addressable storage

Sync events	Cassandra	High-volume time-ordered events per device
Active sessions	Redis	Fast session lookup

Chunked Upload Schema:

```
chunks table:
 chunk_hash   CHAR(64) PRIMARY KEY  (SHA-256 of content)
 storage_key  TEXT                  (S3 object key)
 size_bytes   INTEGER
 ref_count    INTEGER DEFAULT 0  (for deduplication)

file_versions table:
 version_id   UUID PRIMARY KEY
 file_id      UUID REFERENCES files
 chunk_hashes TEXT[]                (ordered list of chunk hashes)
 size_bytes   BIGINT
 created_at   TIMESTAMP
 created_by   UUID
```

Deduplication: If two users upload the same file (identical content), only one S3 object stored. `chunk_hash` is the content fingerprint — if hash exists, reuse the existing S3 object.

5. API Design

```
POST /files/upload/init       → Start chunked upload {file_name, size, chu
PUT  /files/upload/{upload_id}/chunk/{n}  → Upload chunk n
POST /files/upload/{upload_id}/complete   → Finalize upload
GET  /files/{file_id}/download            → Get signed S3 download URL
GET  /files/{file_id}/versions            → List versions
GET  /sync/changes?since={token}          → Get changes since last sync
POST /files/{file_id}/share               → Share file
```

6. High-Level Architecture

```
Upload:
Client → Chunk Splitter (client-side, 4MB chunks)
 ↓ (each chunk)
API → Dedup Check (PostgreSQL chunk_hash) → Skip if exists
 ↓ (new chunk)
S3 (store chunk) → Update chunks table
 ↓ (all chunks uploaded)
API → Create FileVersion (ordered chunk_hash list in PostgreSQL)
 ↓
Kafka → Sync Service → Notify other devices of changes

Download:
Client → API → Fetch chunk_hashes for version
 ↓
Reconstruct S3 keys → Generate signed URLs → Client downloads chunks → R
```

7. How It Works

Upload (chunked): 1. Client splits file into 4MB chunks; computes SHA-256 hash of each. 2. Sends chunk hashes to server → server returns which chunks already exist (dedup). 3. Client uploads only new chunks to S3. 4. After all chunks uploaded, client finalizes → server creates FileVersion record. 5. Kafka event → Sync Service notifies other devices.

Sync (delta sync): 1. Device polls `/sync/changes?since={last_token}` on reconnect. 2. Sync Service returns list of changed files since last sync. 3. Device downloads only changed files/chunks. 4. If file changed locally AND remotely → conflict: both versions kept, user prompted to resolve.

8. Networking Decisions

- **Protocol:** HTTPS; chunked upload via multipart.
- **CDN:** For popular shared files; S3 Transfer Acceleration for upload.
- **WebSocket/SSE:** For push-based sync notifications (instead of polling).
- **Retry:** Chunk upload retried individually on failure (not whole file).

9. Caching

- **File metadata:** Redis, TTL = 10 min.

- **Chunk existence check:** Redis bloom filter → fast "chunk already exists?" check before DB query.
- **Signed download URLs:** Short-lived (1 hour); cached by client.
- **Sync token:** Redis, maps to a Cassandra cursor.

10. Scalability

- **S3:** Infinitely scalable object storage.
- **Metadata DB:** Read replicas; shard by user_id if needed.
- **Dedup:** Content-addressable storage (chunk_hash as key) naturally deduplicates globally.
- **Sync Service:** Stateless; Cassandra for sync events scales horizontally.

11. Reliability & Failure Handling

- **Partial upload:** Client retries individual chunks; idempotent (S3 PUT is idempotent).
- **S3 durability:** 11 nines durability (multiple AZ replication by default).
- **DB failure:** Read replicas; auto-failover; metadata recoverable from S3 object metadata as last resort.
- **Device sync conflict:** Keep both versions; present conflict to user.

14. Trade-offs

Decision	Why	Trade-off
Chunked upload	Resume on failure; parallel uploads; dedup at chunk level	More complex client implementation
Content-addressable chunks	Global dedup; saves massive storage	Hash collision (extremely rare, handled with secondary check)

S3 for binary data	Designed for blobs; infinitely durable	Not suitable for structured queries; need metadata DB
Eventual sync	Performance; devices update asynchronously	Brief inconsistency across devices

Final Interview Answer

> "The key insight for file storage is separating metadata from binary content. Metadata — file names, versions, sharing — goes in PostgreSQL for relational queries. The actual file content goes to S3 for durability and scale. To handle large files, the client splits them into 4MB chunks and computes a SHA-256 hash for each. Before uploading, we check which chunks already exist on the server — this enables delta sync (only upload changed chunks) and global deduplication. For sync across devices, Kafka propagates change events; devices poll a sync API to get changes since their last token. Conflict resolution keeps both versions and prompts the user. This design handles petabytes of storage by leveraging S3's infinite scalability while keeping a lightweight metadata layer in PostgreSQL."

Question 9 — Design a Notification System

What the Interviewer Is Testing

- Multi-channel notification delivery.
- Async architecture.
- Retry and failure handling per channel.
- Rate limiting and user preferences.
- At-least-once delivery guarantees.

1. Clarify Requirements

Functional Requirements

- Send notifications via: Push (iOS/Android), Email, SMS.

- Support user preferences (opt-in/out per channel, per category).
- Templated notifications.
- Retry on failure.

Non-Functional Requirements

- Availability: 99.99%.
- Scale: 100M users, 1B notifications/day.
- Latency: push < 1 second; email < 5 minutes; SMS < 30 seconds.
- At-least-once delivery (prefer duplicate over miss).

2. Assumptions & Scale

```
Notifications/day: 1B → ~11,574/sec
Push: 70%, Email: 20%, SMS: 10%
Push: ~8,100/sec
Email: ~2,315/sec
SMS: ~1,157/sec
```

3. Core Entities

- **Notification:** notif_id, user_id, type, template_id, payload, status, created_at
- **UserPreference:** user_id, channel, category, enabled
- **DeviceToken:** user_id, platform (iOS/Android), device_token, updated_at
- **NotificationTemplate:** template_id, name, subject, body (with variables)

4. Database Decision

Choose: PostgreSQL for preferences/templates + Cassandra for notification log + Redis for dedup.

Why: - Preferences are relational (user → channels → categories). - Notification log: high write volume (11K/sec), append-only → Cassandra. - Dedup: Redis with short TTL prevents sending same notification twice.

5. High-Level Architecture

```
Any Service (Order, Payment, Social)
  ↓ (publish event)
Kafka (topic: notifications)
  ↓
Notification Service (consumer)
  ↓ (check user preferences)
PostgreSQL (preferences) + Redis (device tokens)
  ↓ (route by channel)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■Push Queue (Kafka)■Email Queue   ■SMS Queue ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ↓               ↓               ↓
APNs / FCM     SendGrid/SES    Twilio/SNS
  ↓
Delivery Status → Cassandra (notification log)
```

6. How It Works

1. Source service (e.g., Order Service) publishes event to Kafka.
2. Notification Service consumes event; looks up user preferences in PostgreSQL.
3. If user has opted in for this channel + category → fetch device token (Redis) / email / phone.
4. Render notification using template.
5. Publish to channel-specific queue (Kafka partition by user_id for ordering).
6. Channel workers consume and call external APIs (APNs, FCM, SendGrid, Twilio).
7. On success: log to Cassandra. On failure: retry with exponential backoff; after 3 attempts → DLQ.

7. Retry Strategy

```
Attempt 1: immediate
Attempt 2: 30 seconds later
Attempt 3: 5 minutes later
→ DLQ (manual review / alert)
```

8. Async Processing

- **Fully async** — no notification blocks the source service's response to user.
- Kafka decouples source services from notification delivery.
- Each channel (push, email, SMS) has its own consumer group and queue.

9. Scalability

- **Kafka:** Partition notification topic by user_id for ordered delivery; scale consumer groups.
- **Channel workers:** Independently scalable; add more push workers for more throughput.
- **APNs/FCM/SendGrid:** External APIs have their own rate limits → implement per-channel rate limiting.

10. Reliability

- **Kafka durability:** Messages replicated across brokers; consumers retry from last offset on failure.
- **At-least-once:** Kafka + retry = notifications may be sent twice; client-side dedup via notification_id.
- **External API failure:** Retry logic in channel workers; exponential backoff; DLQ for persistently failed.

14. Trade-offs

Decision	Why	Trade-off
Kafka for async	Decoupled; durable; retryable	Added latency; complex infra
Per-channel queues	Independent scaling; failure isolation	More queues to manage

At-least-once delivery	Better than missing notifications	Duplicate notifications possible

Final Interview Answer

> "A notification system is fundamentally asynchronous — the source service should fire an event and forget. I'd use Kafka as the backbone: source services publish notification events; the Notification Service consumes them, checks user preferences in PostgreSQL, and routes to channel-specific workers for push, email, and SMS. Each channel has its own Kafka partition for independent scaling. External APIs like APNs, FCM, SendGrid, and Twilio handle actual delivery. Failures are handled with retry + exponential backoff + DLQ. The key decisions are: full async for resilience, separate queues per channel for isolation, and at-least-once delivery with client-side deduplication using notification IDs."

Question 10 — Design a Search Autocomplete System

What the Interviewer Is Testing

- Trie data structure knowledge.
- Caching for frequent queries.
- Real-time prefix search at scale.
- Balancing freshness vs speed.

1. Clarify Requirements

Functional Requirements

- As user types, return top 5 autocomplete suggestions.
- Suggestions based on popularity (most searched).
- Near real-time: suggestions updated within 24 hours.

Non-Functional Requirements

- Latency: suggestions returned < 100ms.
- Scale: 10M queries/second (peak — Google scale).

- High availability.

2. Assumptions & Scale

```
QPS: 10M queries/sec
Avg query length: 5 chars → each char triggers a suggestion query
Unique queries: 100M distinct searches/day
Top suggestions: return 5 per prefix
```

3. Core Entities

- **QueryFrequency:** query_string, frequency, updated_at
- **AutocompleteSuggestion:** prefix → [query_string, score] list

4. Database Decision

Choose: Trie in distributed memory (custom service) + Redis for hot prefix cache + Cassandra for raw frequency data.

Why: - Trie enables O(L) prefix lookup (L = length of prefix). - At 10M QPS, in-memory trie per data center is necessary. - Redis caches top-N suggestions for the most common prefixes. - Cassandra stores aggregated query frequencies (updated in batch).

Data flow:

```
User search events → Kafka → Aggregator (hourly batch) → Cassandra
Cassandra → Trie Builder (daily/hourly job) → Distributed Trie (in-memory
Most common prefixes → Redis cache (top 5 suggestions per prefix)
```

5. High-Level Architecture

```
User types "se"
  ↓
Browser (check local browser cache)
  ↓ (miss)
CDN (cache popular prefix responses)
  ↓ (miss)
```

```
API Gateway → Autocomplete Service
  ↓ (check Redis: "se" → [search, see, send, set, sell])
Redis HIT → return suggestions immediately
  ↓ (Redis MISS — rare for short common prefixes)
In-memory Trie → traverse to "se" node → return top 5 children by score
```

6. How It Works

Query flow: 1. User types → browser queries every 100ms (debounced). 2. Autocomplete Service checks Redis for prefix → HIT: return top 5. 3. MISS: traverse in-memory Trie → return top 5 → populate Redis cache.

Update flow: 1. Search events stream to Kafka. 2. Aggregator batches hourly/daily → updates Cassandra query frequency. 3. Trie Builder job rebuilds Trie from Cassandra data (deployed to all Autocomplete Servers). 4. Redis entries for changed prefixes invalidated.

9. Caching

- **CDN:** Cache suggestions for short popular prefixes (1-3 chars) with 5-minute TTL.
- **Redis:** Cache `prefix → [top5 suggestions]` for top 1M prefixes.
- **Local browser cache:** Cache recent prefix responses for 1 minute.

14. Trade-offs

Decision	Why	Trade-off
In-memory Trie	Sub-ms prefix lookup	High memory per server; must fit in RAM
Batch Trie rebuild	Simpler than real-time update	Suggestions may be up to 1 hour stale
CDN for common prefixes	Reduces load to near zero for "a", "th", "se" etc.	Stale suggestions during TTL

Final Interview Answer

> "Autocomplete at 10M QPS can't hit a database on every keystroke — it must be served from memory. I'd use a Trie data structure stored in-memory on each autocomplete server, prebuilt from aggregated search frequency data. The Trie lets us find the top 5 completions for any prefix in O(L) time. Redis caches the results for the most common prefixes so most requests never hit the Trie. CDN caches the shortest prefixes (1-3 chars) which cover the vast majority of traffic. Suggestions are updated hourly: search events flow through Kafka → aggregator → Cassandra → Trie rebuild job. The key trade-off is freshness vs latency — batch rebuilding keeps latency low at the cost of suggestions being up to an hour stale."

SECTION B — NETWORKING-CENTRIC SCENARIOS

Question 11 — Your system has 1M concurrent users and connections are a bottleneck. How would you redesign?

What the Interviewer Is Testing

- Understanding of connection limits and WebSocket scaling.
- HTTP keep-alive and connection pooling.
- Load balancing strategies.
- Horizontal scaling of connection-intensive services.

Answer

Root Causes of Connection Bottleneck:

1. **Too many short-lived connections** → Each HTTP request opens/closes a TCP connection.
2. **Connection limit per server** → Linux default max file descriptors = 65,535 per process.
3. **Stateful connections** → WebSocket or SSE held open per user.

Solutions:

Solution	Description

HTTP Keep-Alive	Reuse TCP connection for multiple HTTP requests; avoid reconnect overhead
HTTP/2 Multiplexing	Multiple logical streams over one TCP connection
Connection Pooling	DB connection pools (PgBouncer); service-to-service pools
Horizontal Scaling	Add more servers; load balance across them
Optimize per-connection cost	Increase OS file descriptor limits; use event-loop architecture (Node.js, Nginx, Go)
Sticky load balancing	WebSocket connections sticky to one server; user-to-server mapping in Redis
Upgrade protocol	gRPC over HTTP/2 for service-to-service → bidirectional streaming, multiplexed

For WebSocket scalability at 1M concurrent connections: - Each server can hold ~100K WebSocket connections (tune OS limits). - 1M connections = 10 servers minimum. - Use Redis pub/sub for cross-server message routing.

Question 12 — Design a CDN

What the Interviewer Is Testing

- CDN architecture and PoP distribution.
- Cache invalidation at global scale.
- Pull vs push CDN.
- Anycast routing.

Answer

Core Design:

```
User (India)
  ↓ DNS lookup
DNS returns nearest CDN PoP IP (using Anycast/GeoDNS)
  ↓
CDN PoP Node (Mumbai)
  ↓ (cache HIT) → return cached content immediately
  ↓ (cache MISS) → fetch from Origin Server → cache → return
```

Pull CDN: CDN pulls content from origin on first request (lazy caching). Simpler but cold start. **Push CDN:** Publisher pushes content to CDN proactively. Better for known static assets.

Cache Invalidation: - TTL-based (expires after fixed time). - Purge API: Explicit invalidation when content changes. - Versioned URLs: `/assets/v2/app.js` → old URL cached separately; new version deployed immediately.

CDN Routing: - **Anycast:** Same IP announced from multiple PoPs; BGP routes to nearest. - **GeoDNS:** DNS returns different IP based on client's geographic location.

SECTION C — CACHING SCENARIOS

Question 13 — Your database is being hammered by 100K reads/sec for user profiles. How do you fix it?

Answer

Diagnosis: Read-heavy system; same data read repeatedly; DB cannot sustain 100K reads/sec efficiently.

Solution: Cache-aside with Redis

```
Request for user_id=123:
 1. Check Redis: GET user:123
```

```
 2. HIT → return cached profile (< 1ms)
 3. MISS → SELECT from PostgreSQL → SET user:123 in Redis (TTL=1h) → retu

Write path (user updates profile):
 → UPDATE PostgreSQL
 → DEL user:123 from Redis  (invalidate; not update)
```

Additional strategies: - **Local L1 cache:** Each API server holds a small in-process LRU cache for the hottest 1K users. - **Staggered TTL:** Add random jitter to TTL (3600 ± 600s) to prevent cache avalanche. - **Read replicas:** For cache misses, read from replica not primary.

What if 1 user is accessed 10K times/second (hot key)? - Replicate the hot key across multiple Redis nodes. - Add local in-process cache on API servers specifically for top N hot users.

SECTION D — MESSAGING SCENARIOS

Question 14 — Design an Order Processing System with Async Notifications

What the Interviewer Is Testing

- Async event-driven architecture.
- Saga pattern for distributed transaction.
- Kafka for reliable event delivery.

Answer

Flow:

```
User places order
  ↓
Order Service → saves order (PostgreSQL, status=PENDING)
  ↓
Kafka: event "order_placed"
```

```
  ↓
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Inventory Service   → reserve items ■
■ Payment Service     → charge card   ■
■ Notification Service → send email   ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ↓ (all succeed)
Kafka: "order_confirmed"
  ↓
Order Service → update status=CONFIRMED
Notification → "Your order is confirmed"
```

Failure Handling (Saga with Compensation):

```
If Payment fails:
 → Kafka: "payment_failed"
 → Inventory Service: release reserved items (compensating transaction)
 → Order Service: update status=FAILED
 → Notification: "Your payment failed"
```

Key Design Decisions: - **Kafka** for reliable delivery; at-least-once guarantees. - **Idempotent consumers** → each service handles duplicate events safely (check if already processed). - **DLQ** for persistently failing events → manual review.

SECTION E — SCALABILITY SCENARIOS

Question 15 — Your system needs to handle a flash sale with 100× normal traffic in 10 minutes

What the Interviewer Is Testing

- Traffic spike handling.
- Inventory consistency under extreme write load.
- Queue-based load leveling.

Answer

Challenges: - 100× normal traffic → servers and DB overwhelmed. - Inventory decrement must be atomic → overselling must not happen. - Most requests will fail (sold out) — wasteful DB hits.

Solutions:

1. Pre-load inventory in Redis:

```
BEFORE sale starts:
SET inventory:product_123 1000
```

```
ON each purchase attempt:
DECR inventory:product_123
If result >= 0 → proceed to payment
If result < 0 → INCR back → return "Sold Out"
```

- DECR is atomic in Redis → no overselling.
- Eliminates DB write for inventory check.

2. Queue-based purchase processing:

```
User "buys" → immediately queued (accepted in < 100ms)
Queue → Purchase Workers → process at sustainable rate
User polled for result or notified when done
```

3. Auto-scale API servers: Pre-warm 10 minutes before sale via scheduled scaling.

4. Rate limiting: Limit each user to 1 purchase attempt per minute to reduce load.

5. Static pre-sale page: CDN-cached countdown page → no dynamic requests until sale opens.

SECTION F — RELIABILITY SCENARIOS

Question 16 — Your primary database fails during peak traffic. What happens?

Answer

With proper design, this is a non-event:

```
Primary DB failure detected by:
 - Load balancer health check
 - Replica monitoring (heartbeat timeout)
 - Application connection error

Auto-failover sequence:
 1. Replica detects primary is unreachable (after 30s timeout)
 2. Replica promotes itself to primary (PostgreSQL Patroni / RDS Multi-AZ)
 3. Load balancer / DNS updated to point to new primary
 4. Application reconnects (connection pooler retries with backoff)
 5. Estimated downtime: 30-60 seconds with Multi-AZ RDS
```

What you should have in place: - **Synchronous replication** to at least one replica (RPO = 0 data loss). - **Automatic failover** (AWS RDS Multi-AZ, PostgreSQL Patroni, MySQL MHA). - **Connection pooler** (PgBouncer) with retry on connection failure. - **Read traffic:** Already going to read replicas — unaffected. - **Circuit breaker:** App stops retrying failed DB queries immediately; returns cached data where possible.

SECTION G — SECURITY SCENARIOS

Question 17 — Design Authentication and Authorization for a Multi-Tenant SaaS System

What the Interviewer Is Testing

- Multi-tenant data isolation.
- Token-based auth vs session auth.
- RBAC design.

- Securing cross-tenant data access.

Answer

Authentication: - **OAuth 2.0 + OIDC** for user login (supports "Login with Google/GitHub"). - Issue **JWT access token** (short TTL: 15 min) + **Refresh token** (long TTL: 7 days, stored in DB for revocation). - API validates JWT signature and claims on every request — stateless.

Multi-Tenant Data Isolation:

Approach	Description	Best For
Row-level tenant_id	All tenants in same DB; filter by tenant_id	Small/medium SaaS
Schema-per-tenant	Separate DB schema per tenant	Medium scale
DB-per-tenant	Complete DB isolation	Enterprise, compliance requirements

RBAC Schema:

```
tenants:   tenant_id, name, plan
users:     user_id, tenant_id, email
roles:     role_id, tenant_id, name (Admin, Editor, Viewer)
user_roles: user_id, role_id
permissions: permission_id, resource_type, action (read/write/delete)
role_permissions: role_id, permission_id
```

Authorization check:

```
JWT contains: {user_id, tenant_id, roles[]}
API request: "can user delete this resource?"
 → Check user's roles (from JWT or cache)
 → Check role has delete permission for this resource_type
```

```
→ Check resource.tenant_id == user's tenant_id (row-level isolation)
```

Security measures: - All API calls require valid JWT. - Every DB query includes `AND tenant_id = {jwt.tenant_id}` — enforced at ORM/query layer. - Rate limiting per tenant (separate limits: free vs paid). - Audit log of all write operations.

SECTION H — REAL-TIME SYSTEMS

Question 18 — Design a Live Location Tracking System (like Uber driver tracking)

(Covered in detail in Question 6 — see Uber design. Key points:)

- **WebSocket** for persistent driver → server location stream.
- **Redis GEO** for storing and querying driver positions.
- **Server-Sent Events (SSE)** or WebSocket for pushing driver location to rider.
- Location updates every 5 seconds; Redis TTL = 30 seconds (auto-expires stale locations).

Question 19 — Design a Real-Time Collaborative Editor (Google Docs)

What the Interviewer Is Testing

- Operational Transformation (OT) or CRDT for conflict resolution.
- WebSocket for real-time sync.
- Persistence strategy.

Answer

Core Challenge: Multiple users editing same document simultaneously — changes must be merged without conflict.

Architecture:

```
User A types → WebSocket → Collaboration Server
Collaboration Server → OT/CRDT merge → apply to document
                     → broadcast change to all connected clients
                     → async persist to DB
```

Conflict Resolution: - **Operational Transformation (OT):** Transform concurrent operations so they can be applied in any order and produce the same result. Used by Google Docs. - **CRDT (Conflict-free Replicated Data Type):** Data structure that automatically merges concurrent changes. Simpler than OT; used by Figma.

Persistence: - Changes buffered in memory; flushed to PostgreSQL every 1 second. - Document stored as: current state (snapshot) + operation log (for undo/history). - S3 for periodic snapshots to avoid replaying entire operation history.

Presence (who's typing where): - WebSocket connected users list in Redis. - Cursor position broadcast to all collaborators every 100ms.

PART C — LLD DESIGN QUESTIONS

LLD Question 1 — Design a Parking Lot System

Requirements

- Multiple floors, multiple parking spots per floor.
- Spot types: Compact, Large, Handicapped, Motorcycle.
- Entry: Vehicle enters → find available spot → assign ticket.
- Exit: Vehicle exits → calculate fee → process payment → release spot.
- Support multiple vehicles: Car, Motorcycle, Truck.

Entities

```
ParkingLot → has many Floors
Floor → has many ParkingSpots
ParkingSpot (spotId, type, isOccupied)
```

```
Vehicle (licensePlate, vehicleType)
Ticket (ticketId, vehicle, spot, entryTime, exitTime, fee)
Payment (amount, method, status)
```

Class Design

```
<<interface>> IVehicle
 getLicensePlate(): String
 getVehicleType(): VehicleType

Car implements IVehicle
Motorcycle implements IVehicle
Truck implements IVehicle

<<enum>> VehicleType { CAR, MOTORCYCLE, TRUCK }
<<enum>> SpotType { COMPACT, LARGE, HANDICAPPED, MOTORCYCLE }

ParkingSpot:
 - spotId: String
 - type: SpotType
 - isOccupied: boolean
 + canFit(vehicle: IVehicle): boolean
 + assign(): void
 + release(): void

Floor:
 - floorNumber: int
 - spots: List<ParkingSpot>
 + findAvailableSpot(vehicleType): ParkingSpot

ParkingLot (Singleton):
 - floors: List<Floor>
 - tickets: Map<String, Ticket>
 + enter(vehicle): Ticket
 + exit(ticket): Payment
```

```
Ticket:
 - ticketId: String
 - vehicle: IVehicle
 - spot: ParkingSpot
 - entryTime: DateTime

FeeCalculator (Strategy Pattern):
 <<interface>> IFeeStrategy
   calculateFee(ticket): double
 HourlyFeeStrategy implements IFeeStrategy
 FlatFeeStrategy implements IFeeStrategy
```

Design Patterns Used

- **Singleton:** ParkingLot (one instance manages entire lot).
- **Strategy:** FeeCalculator (swap fee strategies without changing Ticket/Payment code).
- **Factory:** VehicleFactory (creates correct Vehicle type from string input).

SOLID Applied

- **S:** ParkingSpot handles spot logic; FeeCalculator handles fee logic — separate responsibilities.
- **O:** Add new FeeStrategy without modifying existing FeeCalculator interface.
- **L:** Car, Motorcycle, Truck are interchangeable wherever IVehicle is used.
- **D:** ParkingLot depends on IFeeStrategy interface, not specific implementation.

LLD Question 2 — Design an Elevator System

Requirements

- Multiple elevators in a building.
- Elevator serves floor requests (up/down).
- Optimal elevator dispatching.
- Elevator states: IDLE, MOVING_UP, MOVING_DOWN, MAINTENANCE.

Entities & Classes

```
Building:
 - elevators: List<Elevator>
 - dispatcher: ElevatorDispatcher
 + requestElevator(floor, direction): void

Elevator:
 - id: int
 - currentFloor: int
 - state: ElevatorState
 - destinationFloors: TreeSet<Integer>  (sorted)
 + addDestination(floor): void
 + move(): void
 + openDoor(): void
 + closeDoor(): void

<<enum>> ElevatorState { IDLE, MOVING_UP, MOVING_DOWN, MAINTENANCE }

ElevatorDispatcher:
 + dispatch(floor, direction): Elevator  (finds best elevator)

ElevatorButton:
 + press(floor): void

ExternalButton (on each floor):
 + pressUp(): void
 + pressDown(): void
```

Dispatching Algorithm

```
When floor N requests elevator UP:
 Find nearest elevator that is:
 1. IDLE (closest to floor N)
 2. MOVING_UP and currently below floor N (will pass through)
 3. If none: find any IDLE elevator
```

```
4. Assign floor N to chosen elevator's destination set
```

Design Patterns

- **State:** Elevator behavior changes based on state (IDLE vs MOVING_UP vs MOVING_DOWN).
- **Observer:** Floor buttons notify ElevatorDispatcher; Elevator notifies Building controller on arrival.
- **Strategy:** ElevatorDispatcher uses pluggable dispatching strategy (nearest, least loaded, etc.).

LLD Question 3 — Design a Splitwise (Expense Sharing) System

Requirements

- Users create groups.
- Record expenses within groups.
- Split expenses: equally, by percentage, by exact amount.
- Settle debts.
- Show "who owes whom how much."

Entities

```
User: userId, name, email
Group: groupId, name, members: List<User>
Expense: expenseId, groupId, paidBy: User, amount, splits: List<Split>, de
Split: user: User, amount: double
Balance: Map<userId, Map<userId, double>>  (amount user A owes user B)
```

Class Design

```
<<interface>> ISplitStrategy
 computeSplits(expense, members): List<Split>
```

```
EqualSplit implements ISplitStrategy
PercentageSplit implements ISplitStrategy
ExactSplit implements ISplitStrategy

ExpenseService:
 + addExpense(group, paidBy, amount, strategy, members): Expense
 + settleUp(groupId): Map<User, Map<User, Double>>

BalanceCalculator:
 + calculate(expenses): Map<User, Map<User, Double>>
 + simplifyDebts(balances): List<Transaction>
```

Simplify Debts Algorithm

```
Given: A owes B $10, B owes C $10
Simplified: A pays C $10 directly (B is eliminated)

Algorithm (Greedy):
 Build net balance per person (positive = owed money, negative = owes mone
 Sort creditors (most owed) and debtors (most owed)
 Greedily match largest creditor with largest debtor until all settled
```

Design Patterns

- **Strategy:** Split computation (equal/percentage/exact) — pluggable.
- **Observer:** Notify users when expense added or settled.

LLD Question 4 — Design a Library Management System

Requirements

- Books can be searched by title, author, ISBN.
- Member borrows and returns books.
- Track due dates; calculate fines.

- Limit: max 5 books per member at a time.

Entities

```
Book: ISBN, title, author, publisher, totalCopies, availableCopies
BookItem: bookItemId, ISBN, status (AVAILABLE/BORROWED/RESERVED), conditio
Member: memberId, name, email, borrowedBooks, fineOwed
Librarian: can add/remove books
Borrowing: borrowingId, memberId, bookItemId, borrowDate, dueDate, returnD
```

Class Design

```
<<interface>> ISearchable
 search(query, type): List<Book>

BookCatalog implements ISearchable:
 - books: Map<String, Book>
 + searchByTitle(title): List<Book>
 + searchByAuthor(author): List<Book>
 + searchByISBN(isbn): Book

LibraryMember:
 + borrowBook(bookItem): boolean  (check quota, availability)
 + returnBook(borrowing): double  (returns fine amount)

FineCalculator:
 + calculate(dueDate, returnDate): double  ($1/day overdue)

BorrowingService:
 + checkout(member, bookItem): Borrowing
 + returnBook(borrowing): void
 + getBorrowingHistory(memberId): List<Borrowing>
```

Design Patterns

- **Singleton:** Library (single system instance).

- **Observer:** Notify members when reserved book becomes available.
- **Factory:** SearchStrategyFactory (creates title/author/ISBN search strategy).

PART D — FINAL REVISION

SYSTEM DESIGN INTERVIEW — FINAL REVISION

Must Know Concepts

- HLD vs LLD distinction
- Requirements gathering (functional vs non-functional)
- Back-of-envelope estimation formulas
- CAP theorem and what it means in practice
- Strong vs eventual consistency — when each is required
- Database selection criteria
- ACID vs BASE
- Read/write separation pattern
- Fan-out on write vs fan-out on read
- Idempotency — what it is and why it matters
- Exactly-once, at-least-once, at-most-once semantics

Must Know Components

Component	1-Line Purpose
Load Balancer	Distribute traffic across servers
API Gateway	Single entry point; auth, rate limit, routing

CDN	Serve static assets from edge near user
Redis	In-memory cache; pub/sub; geospatial; distributed lock
Kafka	Durable, high-throughput event streaming
Elasticsearch	Full-text and complex search
S3/Object Storage	Durable binary/blob storage
Circuit Breaker	Stop calling failing service
Service Mesh	Sidecar-based cross-service networking

Must Know Databases

Database	**Best For**
PostgreSQL	ACID transactions, complex queries, financial data
MySQL	General web apps, relational data
Cassandra	High write throughput, time-series, wide-column
MongoDB	Flexible schema, nested documents
Redis	Caching, sessions, queues, pub/sub, geospatial

Elasticsearch	Full-text search, log analytics
DynamoDB	Serverless key-value at massive scale
ClickHouse	Analytics, OLAP, time-series aggregation

Must Know Networking

Concept	**Relevance to System Design**
HTTP/HTTPS	All REST APIs
WebSocket	Real-time chat, live updates, gaming
HTTP/2	Multiplexed APIs, mobile performance
DNS	Load balancing, failover, CDN routing
CDN	Global static asset delivery
TCP keep-alive	Connection reuse; reduce handshake overhead
gRPC	Efficient microservice-to-microservice
TLS	Encryption in transit; always use

Must Know Distributed Systems

Concept	**1-Line**

CAP theorem	CP vs AP — choose based on use case
Consistent Hashing	Minimize data movement when adding nodes
Sharding	Horizontal DB partitioning for scale
Replication	Copies for durability and read scale
Leader Election	Raft/Paxos — one node coordinates
Saga Pattern	Distributed transactions without 2PC
Idempotency	Safe retries; process message only once
Exponential Backoff	Smart retry without hammering failed service

---

Must Know Patterns

HLD Patterns

Pattern	**When**
CQRS	Read and write models have very different needs
Event Sourcing	Audit trail; financial; undo needed
Saga	Multi-service transaction
Circuit Breaker	External service calls

API Gateway	Microservices entry point
Fan-out on Write	Pre-compute feeds for read performance
Strangler	Migrate monolith to microservices

LLD Patterns

Pattern	**Classic Use**
Singleton	DB connection pool, Config
Factory	Create objects without specifying class
Strategy	Pluggable algorithms (sort, payment, fee)
Observer	Event notifications
Decorator	Add behavior without modifying class
State	State machine (order status, elevator)
Command	Undo/redo, task queues

Must Know HLD Questions

1. URL Shortener
2. Rate Limiter
3. Twitter/News Feed
4. WhatsApp/Chat
5. YouTube/Netflix

6. Uber/Maps
7. Google Drive/Dropbox
8. Payment System
9. Notification System
10. Search Autocomplete
11. E-commerce (Amazon/Flipkart)
12. Food Delivery (Swiggy/Zomato)
13. Web Crawler
14. Distributed Cache
15. Distributed Logger

Must Know LLD Questions

1. Parking Lot
2. Library Management
3. Elevator System
4. ATM Machine
5. Splitwise
6. Ride Booking
7. Movie Ticket Booking
8. Chess / Tic-Tac-Toe
9. Snake and Ladder
10. Vending Machine

Most Common Follow-Ups

Follow-up	Standard Answer Direction

"What if DB fails?"	Read replicas + auto-failover + circuit breaker
"What if Redis fails?"	Graceful degradation → fall back to DB
"Traffic spikes 10×?"	Auto-scale (K8s HPA) + CDN + queue-based leveling
"How to prevent double charge?"	Idempotency key + DB UNIQUE constraint
"How to scale the DB?"	Read replicas (read-heavy) → Sharding (write-heavy)
"Why SQL not NoSQL?"	Need ACID / complex joins / structured relational data
"Why Kafka not RabbitMQ?"	Need high throughput / replay / event sourcing
"What if cache is stale?"	Short TTL + active invalidation on write
"How to avoid hot partitions?"	Good shard key selection + consistent hashing
"Multi-region?"	Active-active (complex) or Active-passive (simpler)
"What's the bottleneck?"	Usually the DB — add cache + replicas + sharding
"How to monitor?"	Prometheus (metrics) + Grafana (dashboards) + ELK (logs) + PagerDuty (alerts)

1-Day Revision Order

```
Morning (2 hours):
 1. System Design Approach Framework (5 steps)
 2. Database — SQL vs NoSQL, when to use which
 3. Caching — Cache-aside, Redis, cache problems

Midday (2 hours):
 4. Networking — HTTP/WebSocket/CDN/DNS
 5. Load Balancing — L4 vs L7, algorithms
 6. Messaging — Kafka basics, async patterns

Afternoon (2 hours):
 7. Distributed Systems — CAP, Sharding, Replication
 8. Reliability — Circuit breaker, retry, failover
 9. Security — Auth, JWT, HTTPS, rate limiting

Evening (3 hours):
 10. HLD Patterns — CQRS, Saga, Event-Driven
 11. LLD Patterns — Strategy, Observer, State, Factory
 12. Top 5 HLD Questions (URL Shortener, Twitter, WhatsApp, YouTube, Uber)
 13. Top 3 LLD Questions (Parking Lot, Splitwise, Elevator)
 14. Trade-offs table — SQL vs NoSQL, sync vs async, etc.
```

Interview Communication Template

> "Let me start by clarifying the requirements before jumping into the design.
>
> [Ask 2-3 clarifying questions about scale and core features]
>
> Based on that, my functional requirements are: [list 3-4 core features]. Non-functional requirements: [availability target, latency target, scale].
>
> Let me estimate the scale: [back-of-envelope calculation].
>
> The most important decision in this design is the database. Given [requirement], I'd choose [database] because [reason]. [Explain schema briefly].

> At a high level, the architecture looks like this: [describe client → load balancer → service → cache → DB].
>
> For scaling the reads, I'd add [cache/read replicas]. For async operations like [notifications/analytics], I'd use Kafka.
>
> The main trade-offs in this design are: [list 2-3 explicit trade-offs].
>
> Happy to go deeper on any component — the database, caching strategy, or failure handling."
